PROJECT 9 · BACKEND API

SkillSwap

Student Skill Sharing Platform

A step-by-step guide for our group presentation. We start with the problem statement, break it into parts, show how the whole app works in Swagger, and then explain the code for each part in simple words.

Section	What you will learn
1. The problem	What the problem statement asks, the 3 roles, and its 12 components
2. The workflow in Swagger	The full story of a mentor and a student, step by step, with real screenshots
3. The code, part by part	For every component: the code and a simple explanation of each line
4. The 16 requirements	Each technical requirement: expected, how we implemented it, code and proof
5. Presentation plan	Who presents what, what to say, and likely questions
Appendix	How to run the project, and fixes for common problems

Member	Name	Presents
Member 1	_____	Section 1 – the problem, roles, components · Requirements 1, 2
Member 2	_____	Section 2 – live workflow in Swagger · Requirements 11, 15
Member 3	_____	Section 3.1–3.2 – database and roles · Requirements 3, 4, 14
Member 4	_____	Section 3.3–3.7 – login, security, skills, search, admin · Requirements 6–10
Member 5	_____	Section 3.8–3.13 – booking, cancel, history, reviews, validation, testing · Requirements 5, 12, 13, 16

Tech: FastAPI · PostgreSQL · SQLAlchemy · Pydantic · JWT · bcrypt · Swagger · Postman

SECTION 1

The Problem Statement

What we were asked to build, who uses it, and the parts it is made of

1.1 The problem statement (as given)

> **9. SkillSwap — Student Skill Sharing**
>
> **Suggested Roles:** Student/Mentor, Admin
> **Core Features:** Create skill profiles • Search skills/mentors • Book mentoring sessions • Cancel sessions • Session history • Reviews and ratings
> **Minimum API expectation:** Authentication APIs + CRUD APIs + protected APIs + role-based APIs + search/filter APIs + validation/business-rule handling.

1.2 In simple words

In every college, some students are very good at something — Python, Excel, guitar, design — and other students want to learn exactly that. Today they find each other by luck. **SkillSwap connects them.**

- A student who can teach becomes a **mentor** and publishes what they teach (a **skill profile**).
- A student who wants to learn **searches** for a skill or a mentor, **books** a time, attends the session, and then gives a **rating**.
- Either of them can **cancel** before the session, and both can see their **session history**.
- An **admin** looks after the platform: manages accounts and watches what is happening.

1.3 What exactly did we build? (Backend = API)

We built the **backend** only: the part that runs on the server, checks the rules and saves data in the database. A website or mobile app would talk to it, but for this project we test it with **Swagger** (a web page FastAPI creates for us) and **Postman**.
What is an API? Think of a restaurant. You (the app) never go into the kitchen (the database). You give your order to the waiter (the API) using a fixed menu (the endpoints, like `POST /sessions`). The waiter checks the order, the kitchen prepares it, and the waiter brings back the result (a JSON response with a status code like `201 Created`).

```
one API call, in plain text

Request  →  POST http://127.0.0.1:8000/sessions          (method + URL = "what I want")
            Authorization: Bearer eyJhbGciOi...          (my login token = "who I am")
            {"skill_id": 5, "scheduled_at": "2026-10-05T10:00:00Z", "duration_minutes": 60}

Response ←  201 Created
            {"id": 5, "status": "booked", "mentor_id": 7, "student_id": 8, ...}
```

1.4 Who are the users? (the 3 roles)

The problem says “Student/Mentor, Admin”. The technical requirements ask for **at least 3 roles**, so we made Student and Mentor two separate roles. In real life a mentor is also a college student; in our app the **role decides what the person is allowed to do**.

Student	
Who	A learner. Example: **Rahul**, a 2nd-year student who wants to learn Python.
Can do	Search skills and mentors · book a session · cancel own session · see own session history · write, edit and delete own reviews
Cannot do	Create skills · complete sessions · see other people's sessions · use admin APIs
Account	Signs up with `"role": "student"` (this is the default)

Mentor	
Who	A teacher. Example: **Priya**, a final-year student who knows Python well.
Can do	Create, edit and delete own skill profiles · see sessions booked with them · mark a session as completed · cancel own sessions
Cannot do	Book sessions · write reviews · edit another mentor's skills · use admin APIs
Account	Signs up with `"role": "mentor"`

Admin	
Who	The platform manager (for example a teacher or the college IT team).
Can do	List and search all users · change a user's role · deactivate / activate / delete accounts · see all sessions · delete sessions and reviews · see statistics
Cannot do	Change or delete their own account (protection against locking yourself out)
Account	**Cannot sign up.** One admin is created automatically from the `.env` file when the server starts

1.5 The full journey at a glance

This is the order in which things happen. Section 2 shows each of these steps live in Swagger.

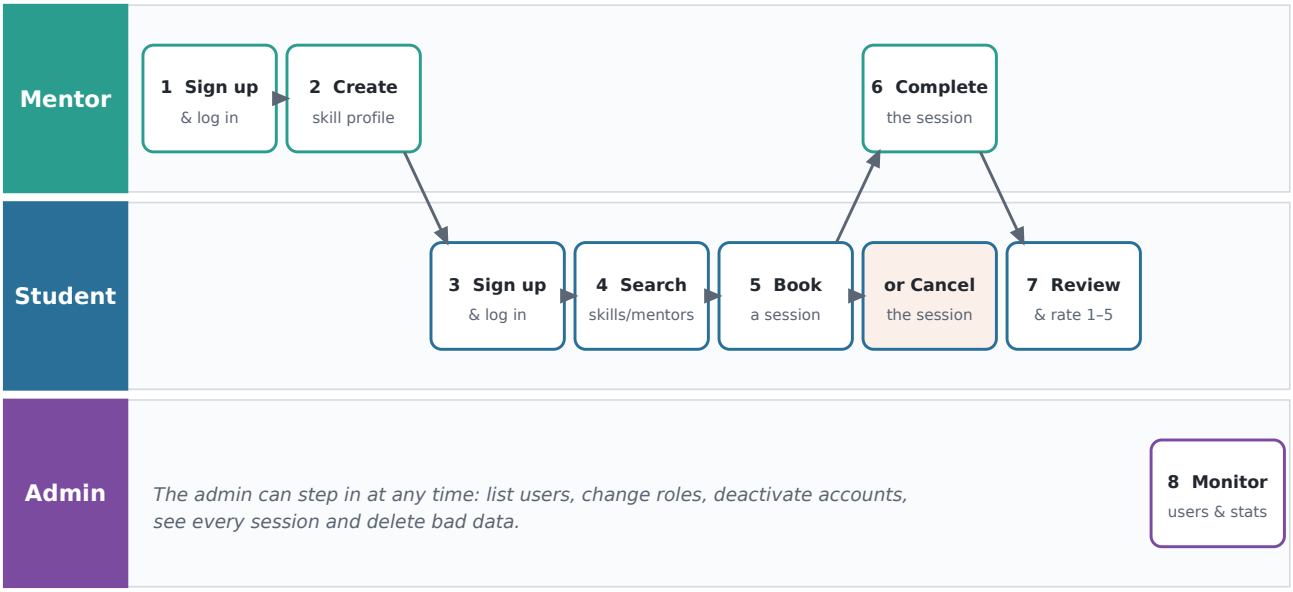


Figure. Who does what, from left (first) to right (last).

1.6 Breaking the problem into components

We split the problem statement into **12 components**: the 6 core features plus the 6 API expectations. Every component became one or more API endpoints. Section 3 explains the code of each one.

#	Component (from the problem)	What we built	Code section
C1	Create skill profiles	POST / PUT / DELETE /skills (mentor only, own skills)	3.5
C2	Search skills / mentors	GET /skills?q=&category;=&level;=, GET /users/mentors?skill=&min;_rating=	3.6
C3	Book mentoring sessions	POST /sessions (student), with a no-double-booking rule	3.8
C4	Cancel sessions	PATCH /sessions/{id}/cancel (only before the start time)	3.9
C5	Session history	GET /sessions?status=&from;_date=&to;_date=	3.10
C6	Reviews and ratings	POST/GET/PUT/DELETE /reviews + average rating per mentor	3.11
E1	Authentication APIs	POST /auth/register, POST /auth/login (JWT token)	3.3
E2	CRUD APIs	Create/Read/Update/Delete for users, skills, sessions, reviews	3.5–3.11
E3	Protected APIs	Private endpoints need a valid token, otherwise 401	3.4
E4	Role-based APIs	Some endpoints only for student / mentor / admin, otherwise 403	3.4, 3.7
E5	Search / filter APIs	Query parameters: q, category, level, status, dates, rating, skip, limit	3.6, 3.10
E6	Validation / business rules	Pydantic checks (422) + rules like no double booking (400/409)	3.12

Underneath all components are two shared building blocks: the **database** (section 3.1) and the **roles** (section 3.2).

SECTION 2

The Workflow in Swagger

The complete story of Priya (mentor) and Rahul (student), shown live in the real API

2.1 What is Swagger and how to use it

FastAPI automatically creates a web page with every endpoint of our API. Start the server and open `http://127.0.0.1:8000/docs`.

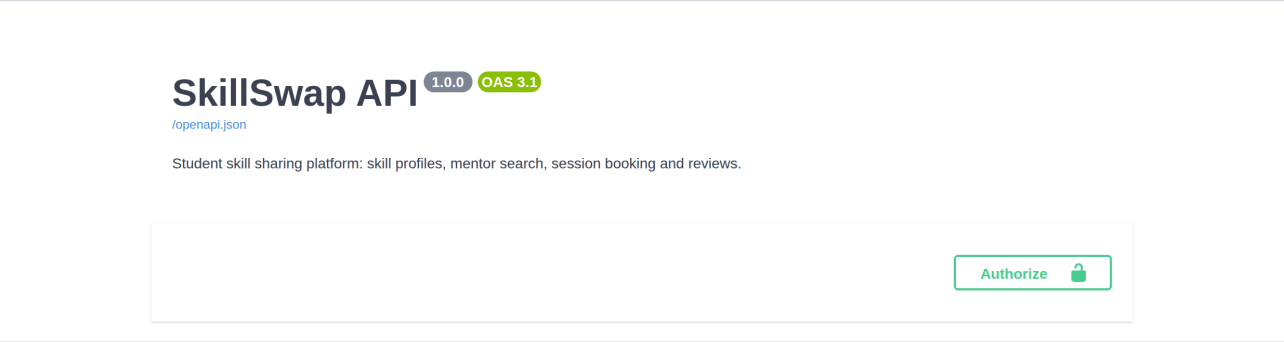


Figure 1. The top of the Swagger page: title, description and the green Authorize button.

Below it, every endpoint is listed, grouped by topic. The colour shows the HTTP method: **POST** = create, **GET** = read, **PUT** / **PATCH** = update, **DELETE** = delete. A **lock icon** means you must be logged in.

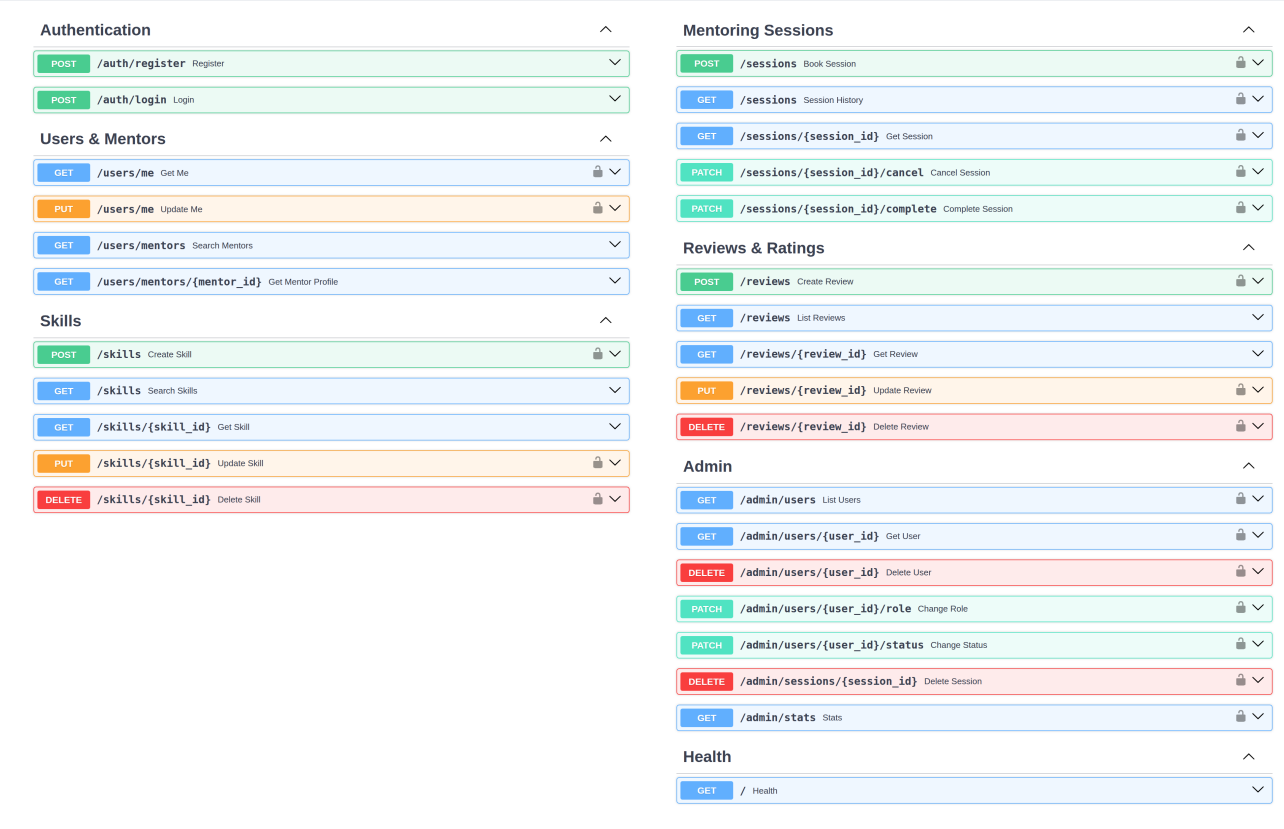


Figure 2. All 29 endpoints in Swagger, grouped by topic.

How to call any endpoint

1. Click the endpoint to open it.
2. Click **Try it out**.
3. Fill in the request body (JSON) or the parameters.
4. Click **Execute**.
5. Read the **Code** (for example 200, 201, 403) and the **Response body** below it.

2.2 The story

We will follow two people. **Priya Sharma** is a final-year student who is good at Python, so she joins as a **mentor**. **Rahul Verma** is a second-year student who wants to learn Python, so he joins as a **student**. At the end the **admin** checks the platform. Every screenshot below is a real request to our running API with PostgreSQL.

STEP 1 **MENTOR** **Priya creates her account**

Endpoint	`POST /auth/register`	**Login needed**	No

Priya sends her name, email, password, role and a short bio. The server checks the data, hashes the password and saves her in the `users` table.

- **Notice:** the code is **201 Created**, and Priya got user id **7**.
- **Notice:** the response does **not** contain the password — we never send it back.

POST /auth/register Register

Parameters

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "name": "Priya Sharma",
  "email": "priya.mentor@skillswap.com",
  "password": "Priya1234",
  "role": "mentor",
  "bio": "Final-year CS student, 3 years of Python experience"
}
```

Request URL

```
http://127.0.0.1:8000/auth/register
```

Server response

Code	Details
201	Response body

```
{
  "id": 7,
  "name": "Priya Sharma",
  "email": "priya.mentor@skillswap.com",
  "role": "mentor",
  "bio": "Final-year CS student, 3 years of Python experience",
  "is_active": true,
  "created_at": "2026-10-02T13:07:01.278328+05:30"
}
```

Figure 3. Registering a mentor.

STEP 2 **ANYONE** **Trying a private page without logging in**

Endpoint	`GET /users/me`	**Login needed**	Yes (none sent)

Before logging in, we try to read “my profile”. This endpoint is **protected**.

- **Notice:** the API answers **401 Unauthorized** — “Invalid or missing token”.

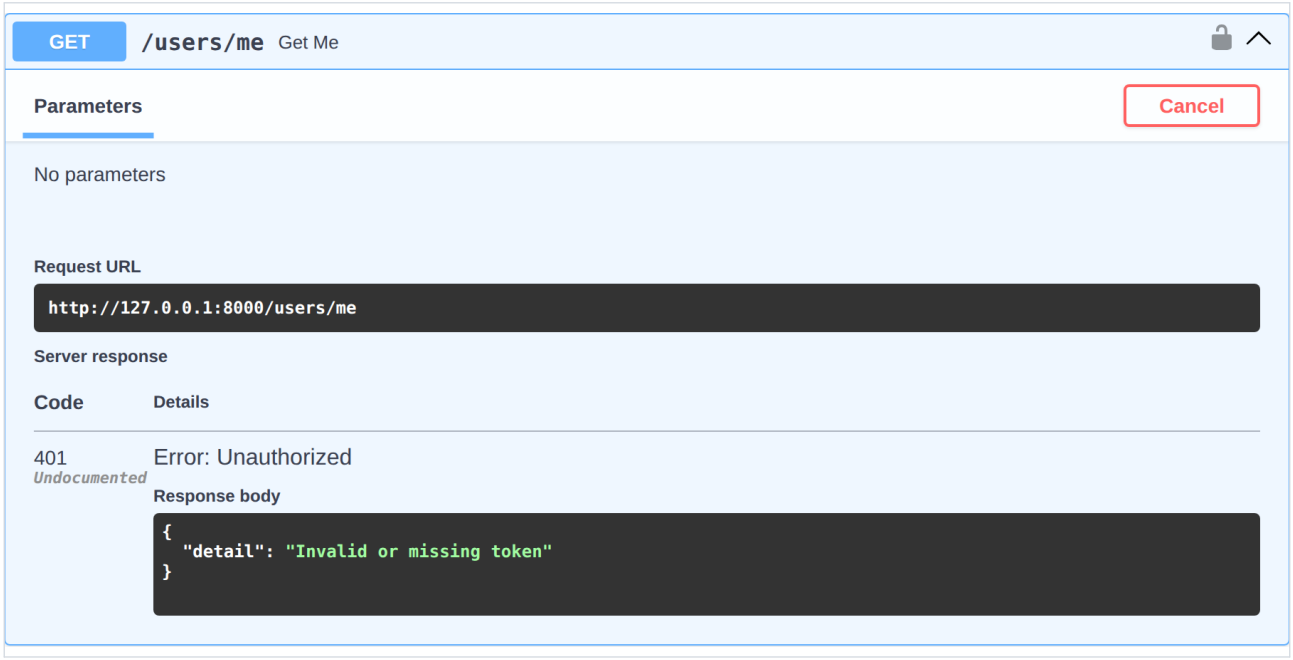


Figure 4. A protected endpoint refuses a request without a token.

STEP 3 **MENTOR** **Priya logs in**

Endpoint	`POST /auth/login`	**Login needed**	No

Priya sends her email and password. If they are correct, the server gives her an **access token** (a JWT). It is like a visitor badge: she shows it with every request and it is valid for 60 minutes.

- **Notice:** the response has `access_token`, `"token_type": "bearer"` and her role.

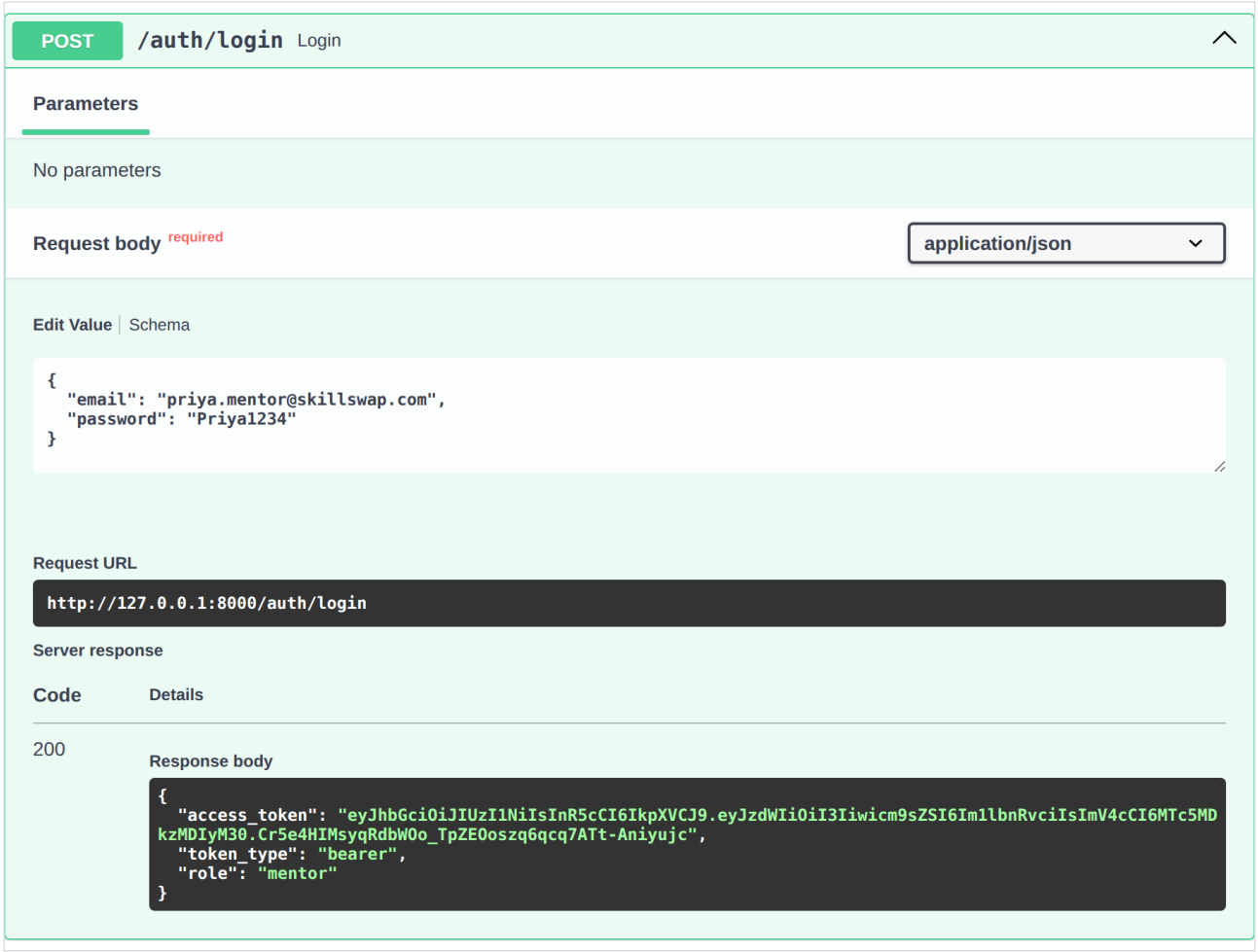


Figure 5. Logging in returns a JWT access token.

STEP 4 **MENTOR** **Priya gives the token to Swagger**

Endpoint	`Authorize button (top right)`	**Login needed**	—

We copy the `access_token`, click **Authorize**, paste it and click Authorize. From now on Swagger adds `Authorization: Bearer <token>` to every request automatically.

- **Notice:** the lock icons in the endpoint list now look closed.

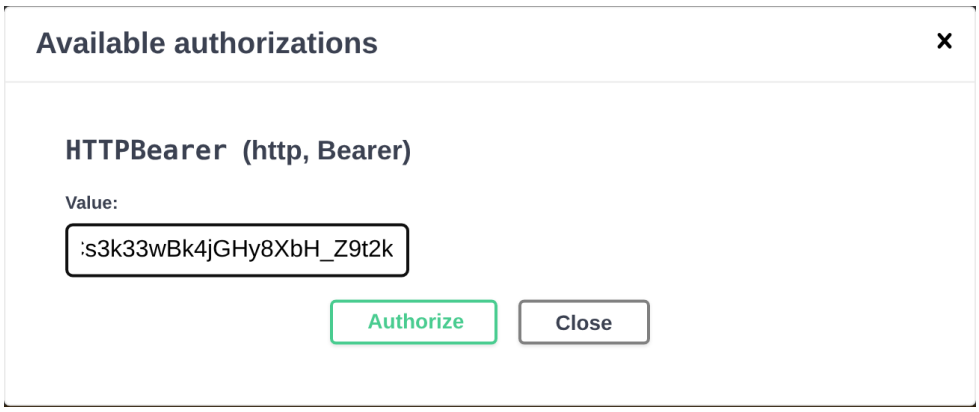


Figure 6. The Authorize dialog with the token pasted in.

STEP 5 **MENTOR** **Now the private page works**

Endpoint	`GET /users/me`	**Login needed**	Yes

The same request as step 2, but now with the token.

- **Notice:** this time the code is **200 OK** and we see Priya's profile with role `mentor`.

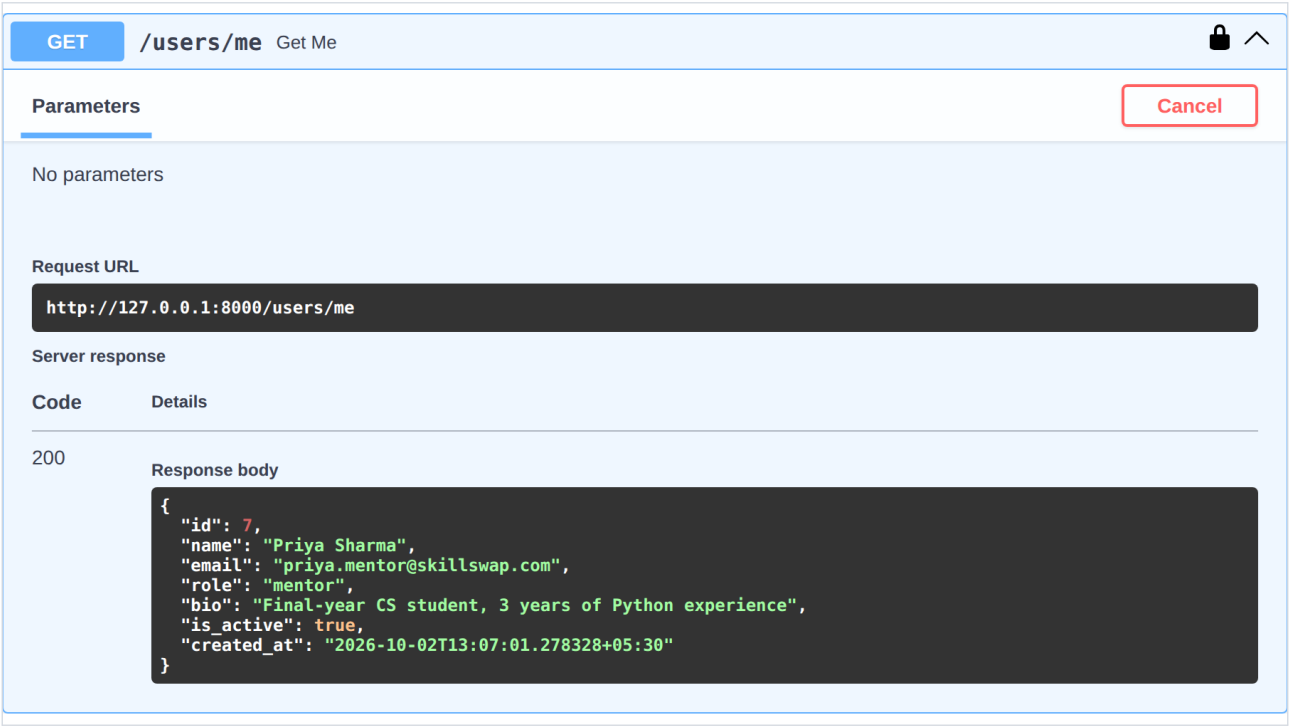


Figure 7. The protected endpoint works with a valid token.

STEP 6 MENTOR Priya creates her skill profile (C1)

Endpoint	`POST /skills`	**Login needed**	Yes – mentor only

Priya publishes what she teaches: a title, a category, a level (beginner / intermediate / advanced) and a description. In the same way she also added a second skill, “Data Analysis with Pandas”.

- **Notice: 201 Created**, skill id **5**, and `"mentor_id": 7` is filled in from her token, not typed by her.

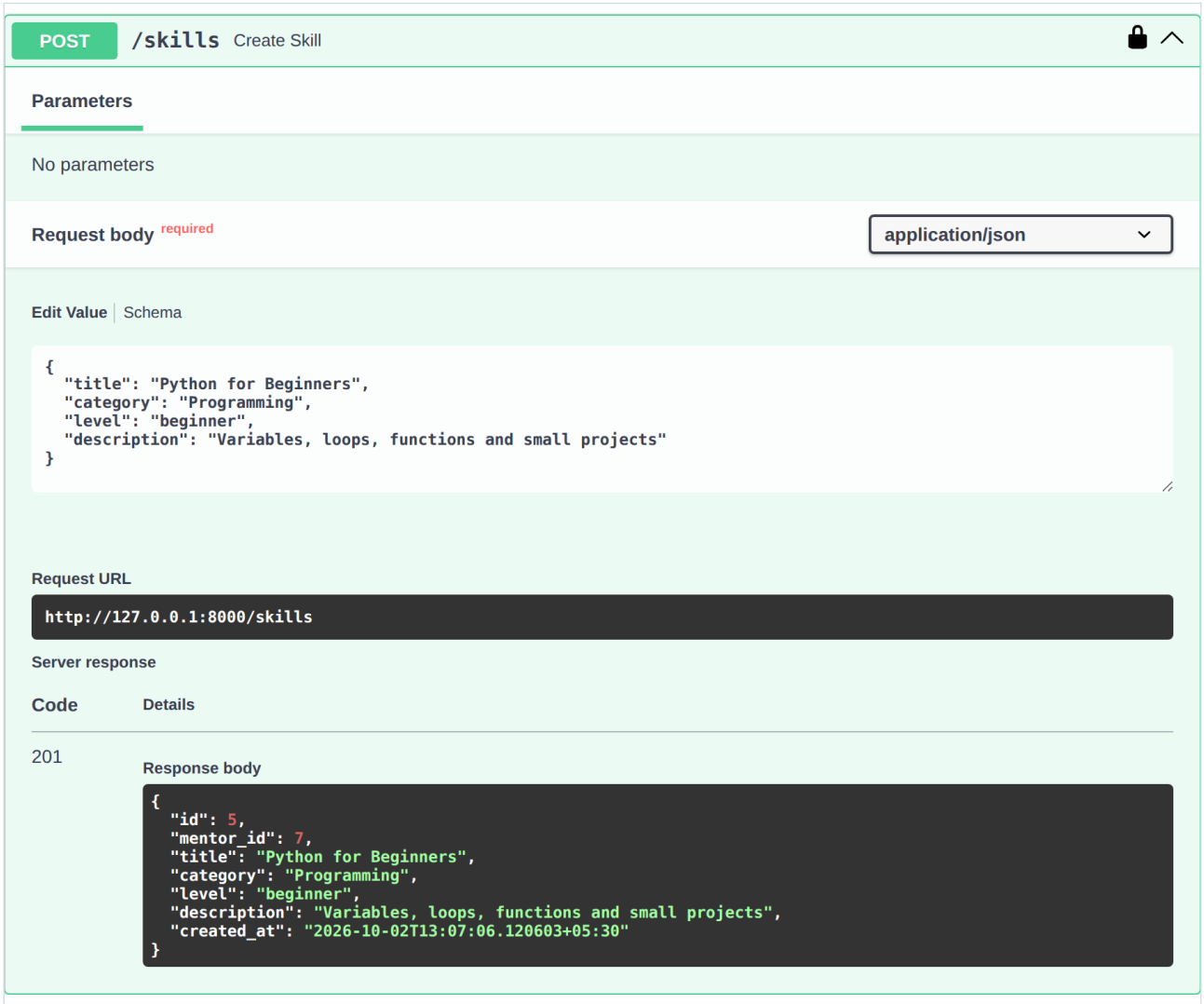


Figure 8. A mentor creates a skill profile.

STEP 7 STUDENT Rahul creates his account

Endpoint	POST /auth/register	Login needed	No

Rahul registers with `"role": "student"`. He then logs in and authorizes Swagger exactly like Priya did in steps 3 and 4.

- **Notice:** Rahul got user id **8**.

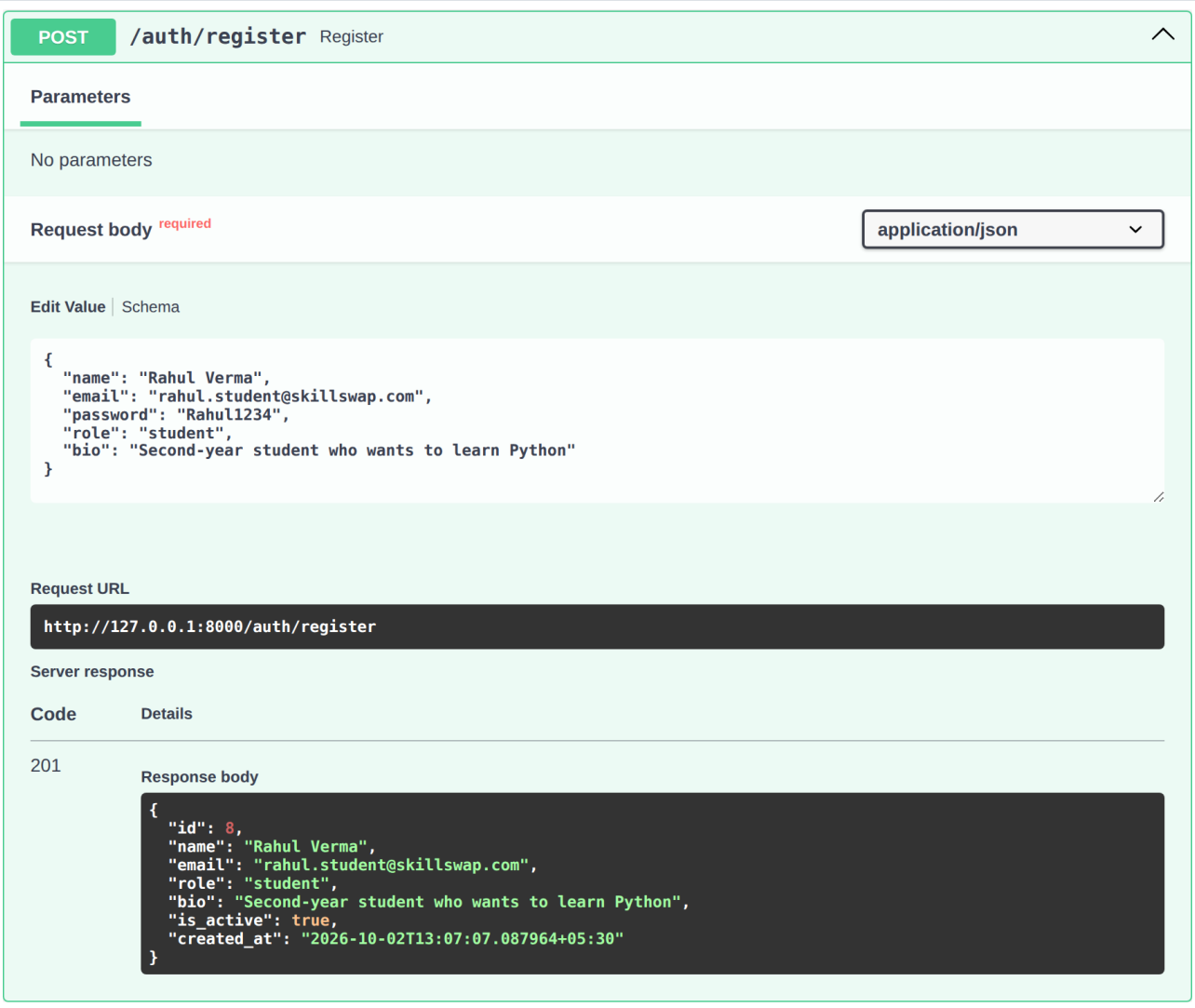


Figure 9. Registering a student.

STEP 8 **STUDENT** **Rahul tries to create a skill — not allowed**

Endpoint	`POST /skills`	**Login needed**	Yes – mentor only

Creating skills is only for mentors. This shows our **role-based** protection.

- **Notice:** Rahul **is** logged in, but the answer is **403 Forbidden** — "You do not have permission".
- **Notice:** 401 = "we don't know who you are"; 403 = "we know you, but you are not allowed".

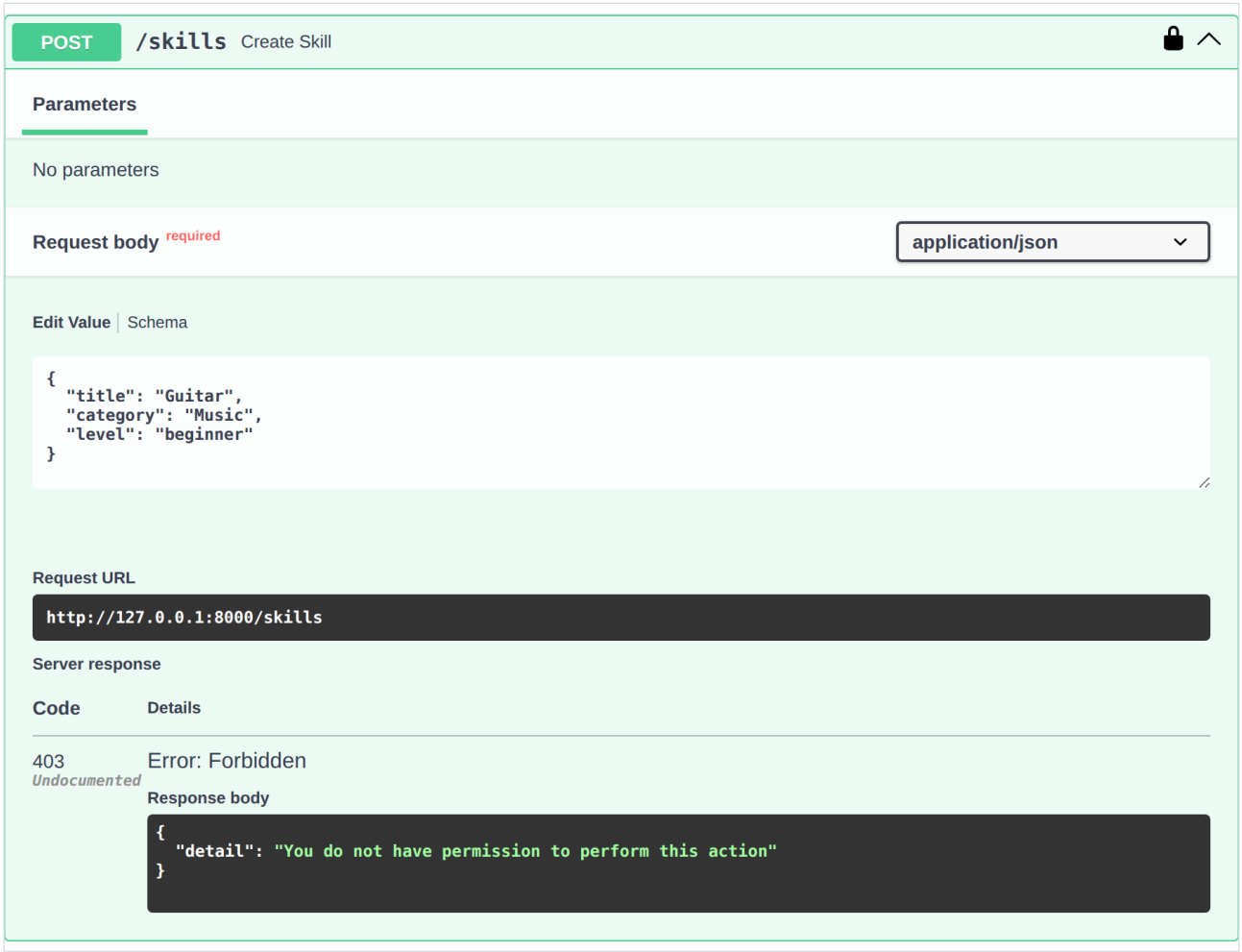


Figure 10. A student is blocked from a mentor-only endpoint.

STEP 9 STUDENT Rahul searches for skills (C2)

Endpoint	GET /skills?q=python&level=beginner	Login needed	No

Rahul types “python” in q and picks level “beginner”. These are **query parameters** (the part after ? in the URL). Search is public so people can browse before signing up.

- **Notice:** the Request URL shows the filters, and the result is Priya’s “Python for Beginners”.

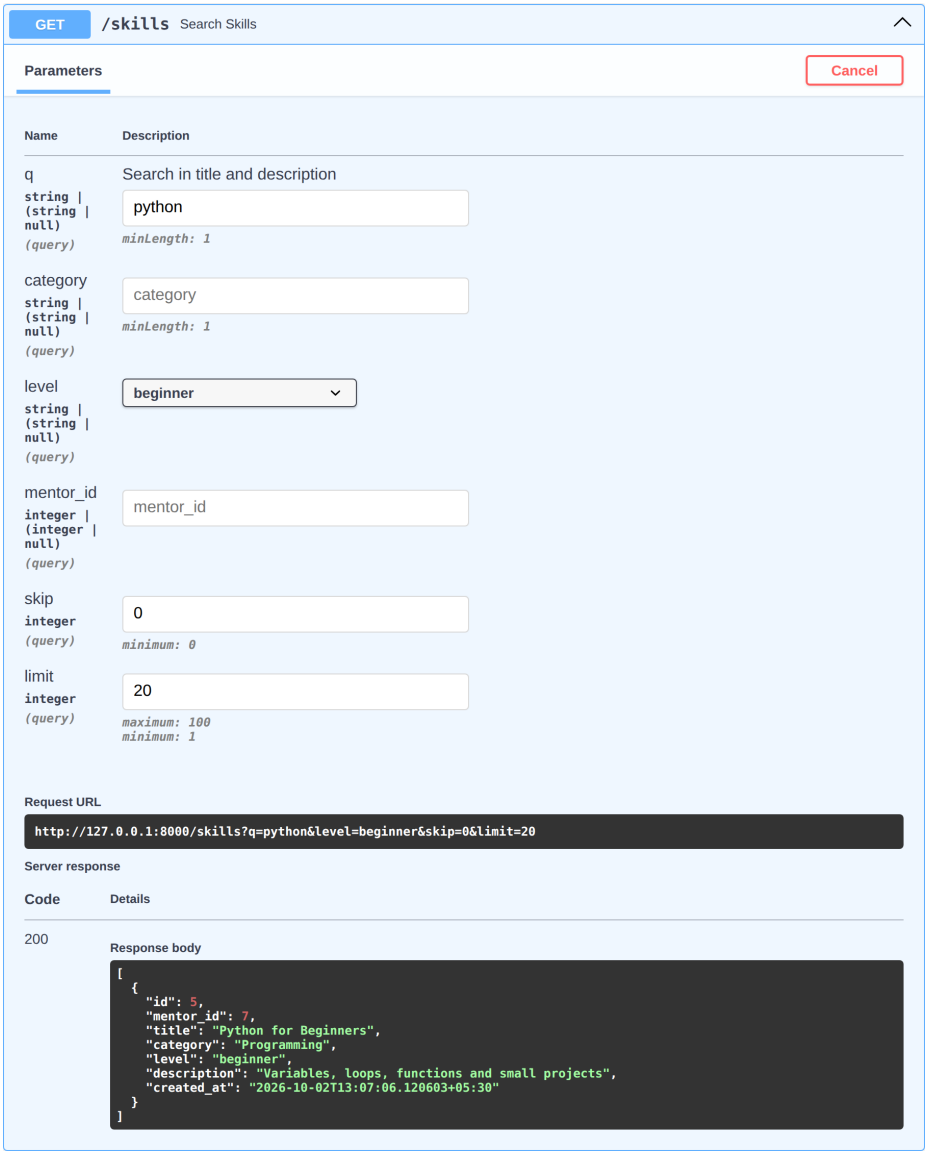


Figure 11. Searching skills with filters.

STEP 10 **STUDENT** **Rahul searches for mentors (C2)**

Endpoint	`GET /users/mentors?skill=python`	**Login needed**	No

Rahul can also search for mentors who teach a skill, and filter by minimum rating.

- **Notice:** Priya is found. `"average_rating": null` and `"review_count": 0` because nobody has reviewed her yet.

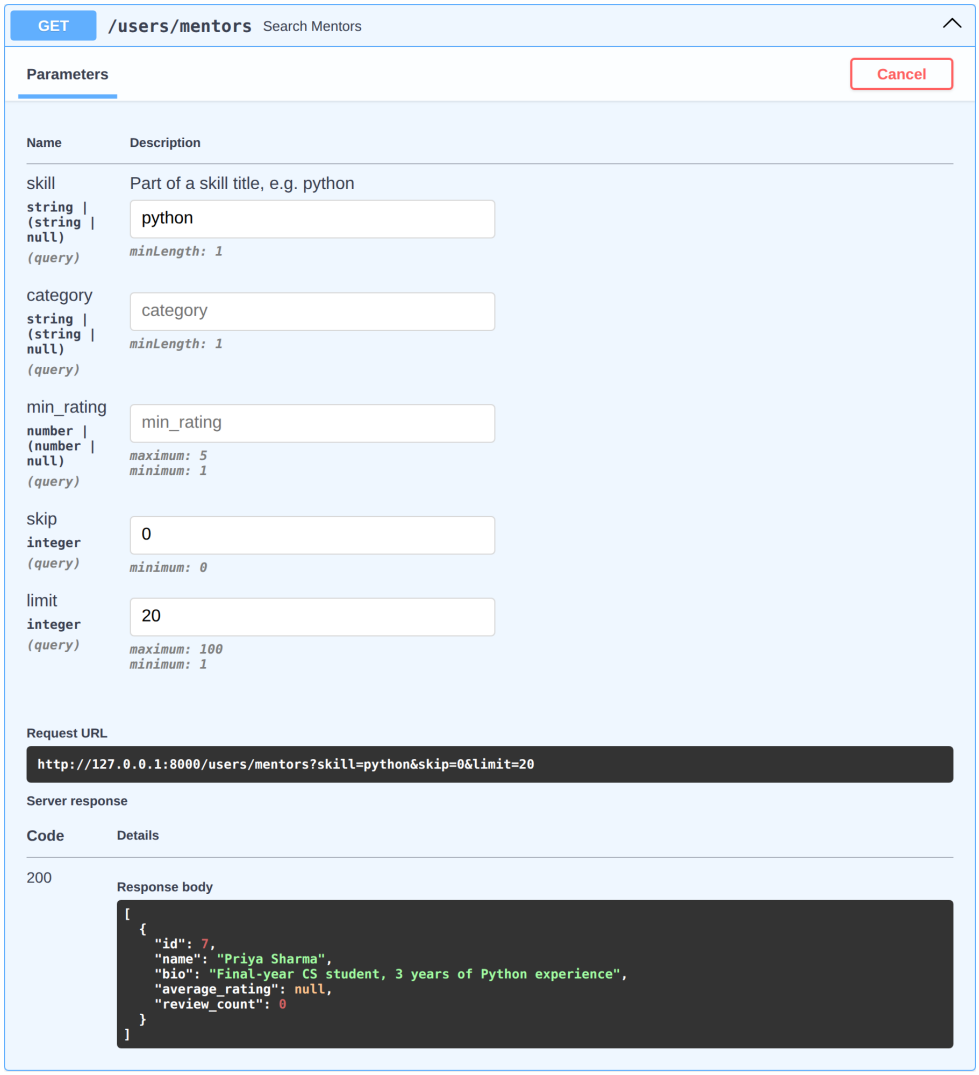


Figure 12. Searching mentors by skill.

STEP 11 STUDENT Rahul books a session (C3)

Endpoint	POST /sessions	Login needed	Yes – student only

Rahul books skill 5 for 60 minutes and writes what he needs help with.

- **Notice: 201 Created**, session id **5**, status `booked`, and the mentor is found automatically from the skill.
- **Notice:** we sent `10:00:00Z` (UTC). The server shows `15:30:00+05:30` — the same moment in Indian time.

POST /sessions Book Session

Parameters

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "skill_id": 5,
  "scheduled_at": "2026-10-05T10:00:00Z",
  "duration_minutes": 60,
  "notes": "I want help with loops and functions"
}
```

Request URL

```
http://127.0.0.1:8000/sessions
```

Server response

Code Details

201

Response body

```
{
  "id": 5,
  "student_id": 8,
  "mentor_id": 7,
  "skill_id": 5,
  "scheduled_at": "2026-10-05T15:30:00+05:30",
  "duration_minutes": 60,
  "status": "booked",
  "notes": "I want help with loops and functions",
  "created_at": "2026-10-02T13:07:11.521479+05:30"
}
```

Figure 13. Booking a mentoring session.

STEP 12 **STUDENT** **Booking the same time again — blocked (business rule)**

Endpoint	`POST /sessions`	**Login needed**	Yes – student only

We try to book Priya again at the same time. A mentor cannot be in two sessions at once.

- **Notice: 409 Conflict** — “Mentor already has a session booked at this time”.

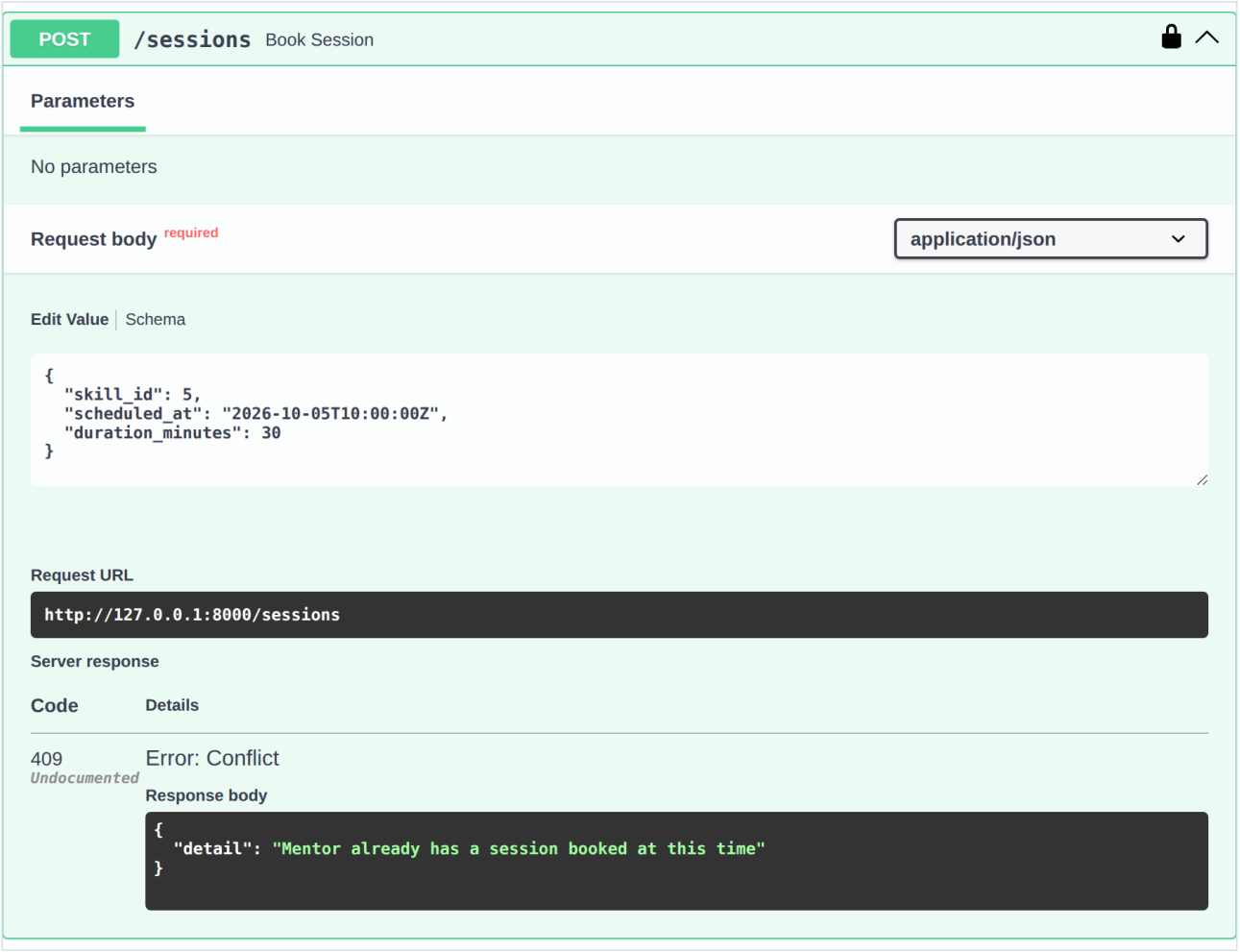


Figure 14. Double booking is not allowed.

STEP 13 **MENTOR** **Priya sees her upcoming sessions (C5)**

Endpoint	`GET /sessions?status=booked`	**Login needed**	Yes

Logged in as Priya, we ask for her sessions with status “booked”. She only sees sessions where she is the mentor (an admin would see all sessions).

- **Notice:** Rahul’s booking (student_id 8) appears in Priya’s list.

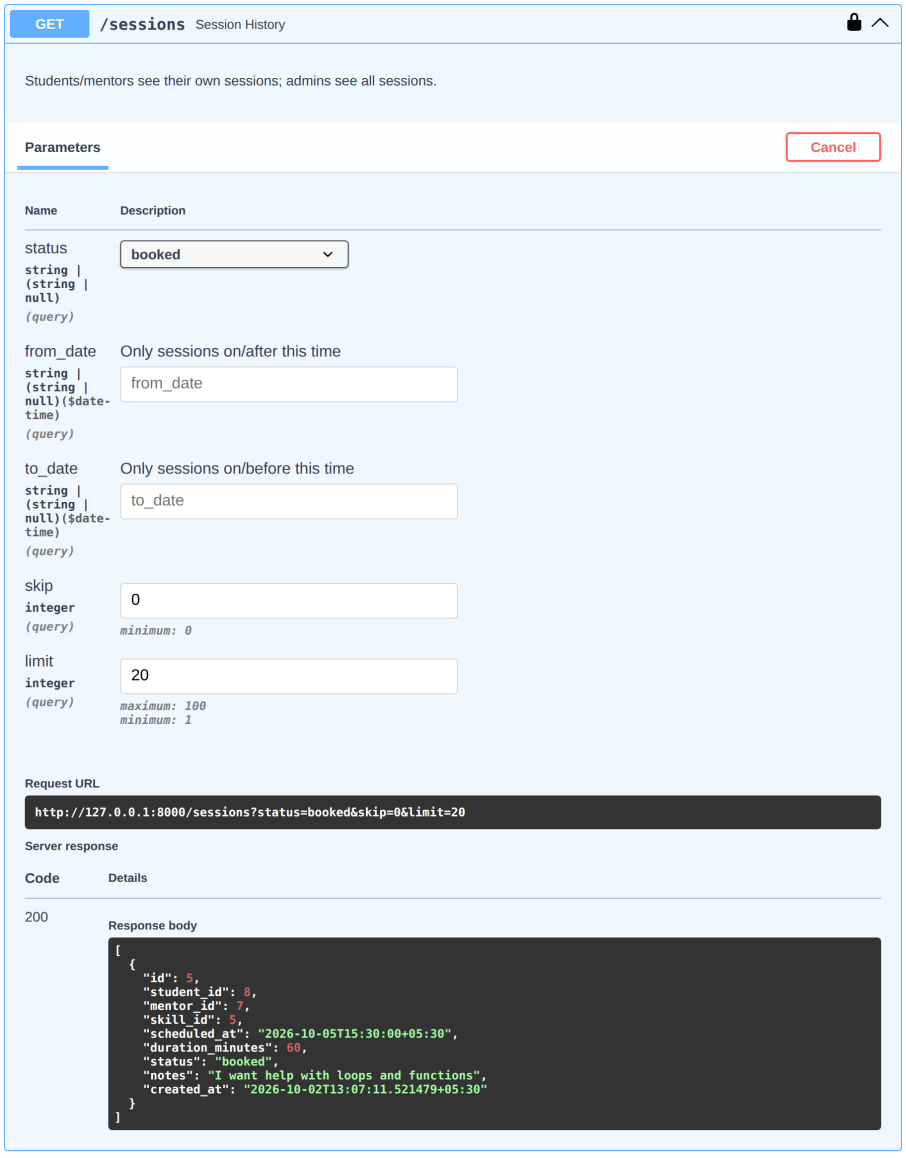


Figure 15. A mentor’s upcoming sessions.

STEP 14 **MENTOR** **Priya marks the session as completed**

Endpoint	`PATCH /sessions/5/complete`	**Login needed**	Yes – mentor / admin

After teaching, Priya marks session 5 as done. 5 is a **path parameter** (part of the URL itself).

- **Notice:** status changed from `booked` to `completed`.

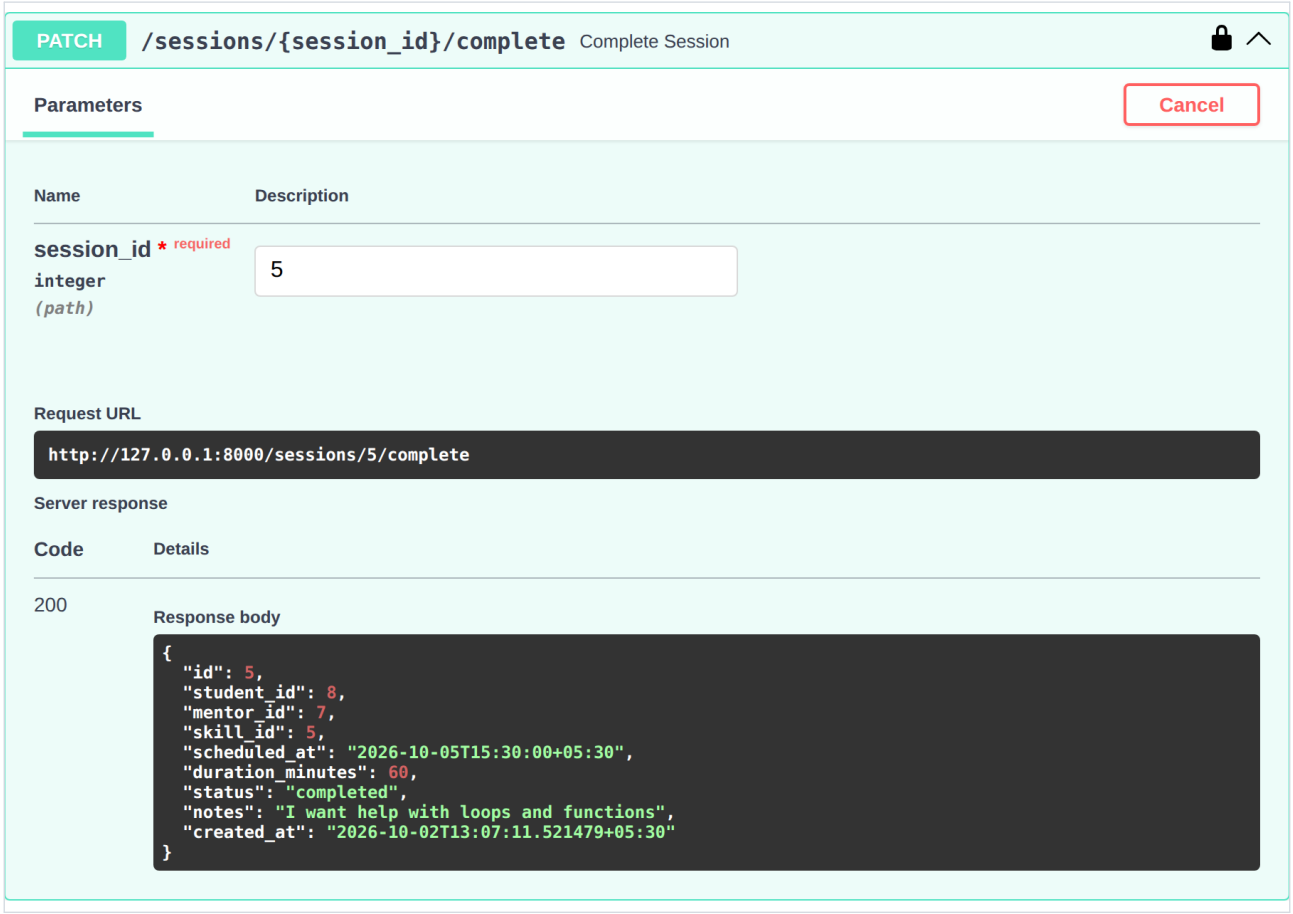


Figure 16. Completing a session.

STEP 15 STUDENT Rahul reviews the session (C6)

Endpoint	POST /reviews	Login needed	Yes – student only

Only the student of a **completed** session can review it, and only once. Rahul gives 5 stars.

- **Notice: 201 Created**. A second review for the same session would get 409; a rating of 6 would get 422.

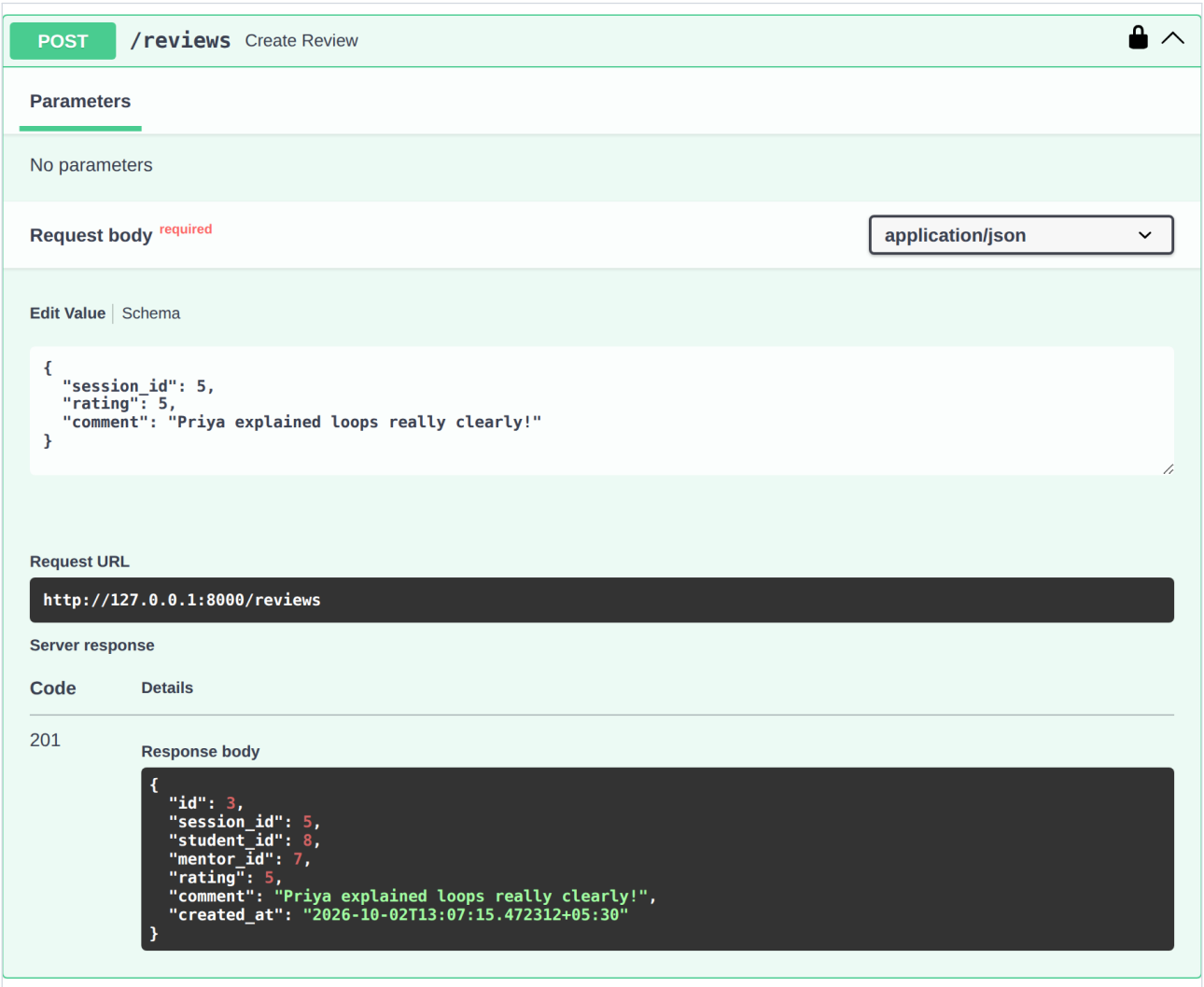


Figure 17. Writing a review.

STEP 16 **ANYONE** **Priya’s public profile now shows her rating (C6)**

Endpoint	`GET /users/mentors/7`	**Login needed**	No

Anyone can open a mentor’s profile. The average rating is calculated by the database from all her reviews.

- **Notice:** `"average_rating": 5`, `"review_count": 1`, and both of her skills are listed.

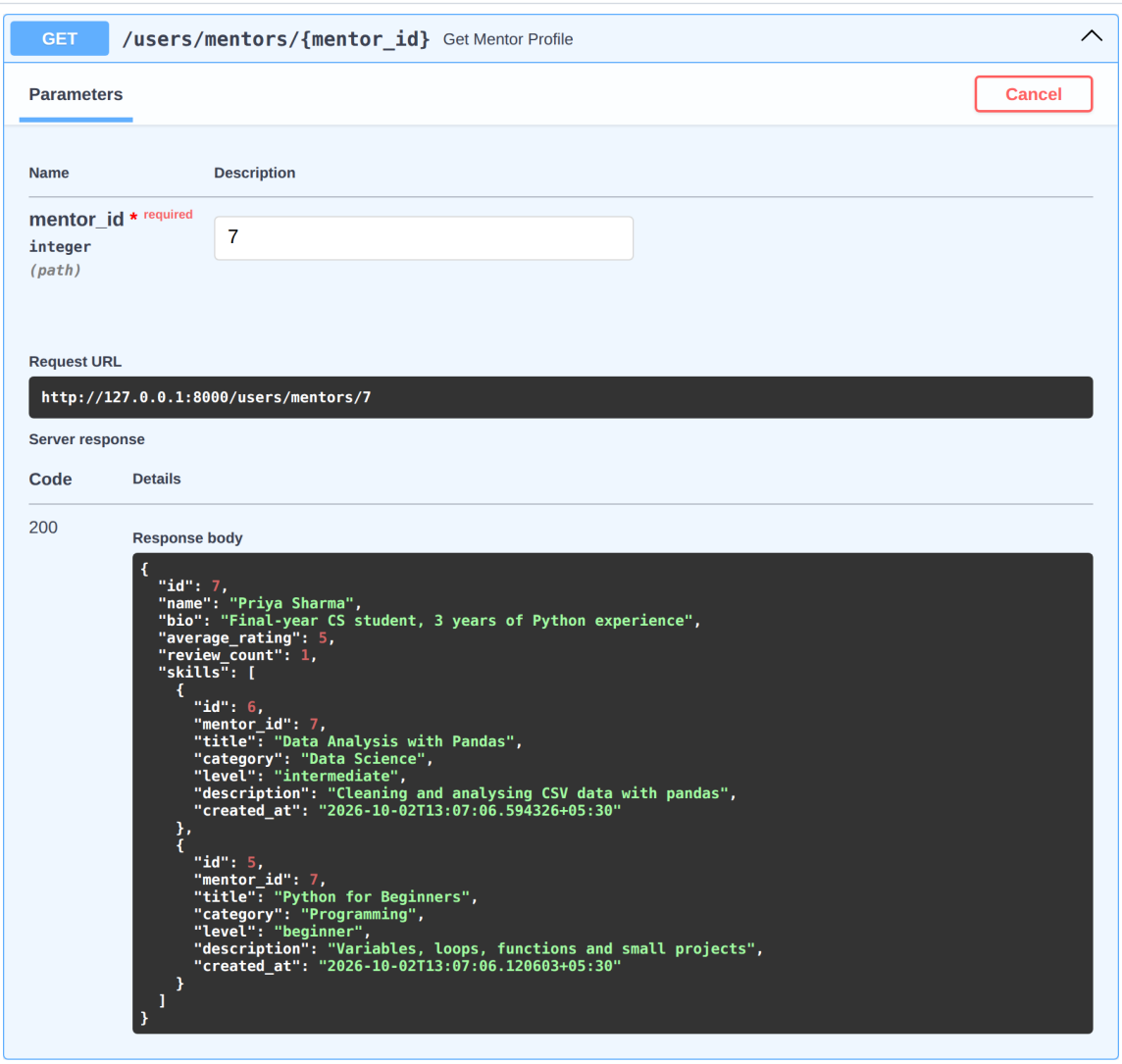


Figure 18. Mentor profile with skills and average rating.

STEP 17 **STUDENT** **Rahul books another session and cancels it (C4)**

Endpoint	`PATCH /sessions/6/cancel`	**Login needed**	Yes

Rahul booked a second session (id 6) for two days later, then changed his mind. The student or the mentor can cancel, but only **before the session starts** and only if it is still booked.

- **Notice:** status is now `cancelled`. We keep the row so it still appears in the history. Cancelling it again gives 400.

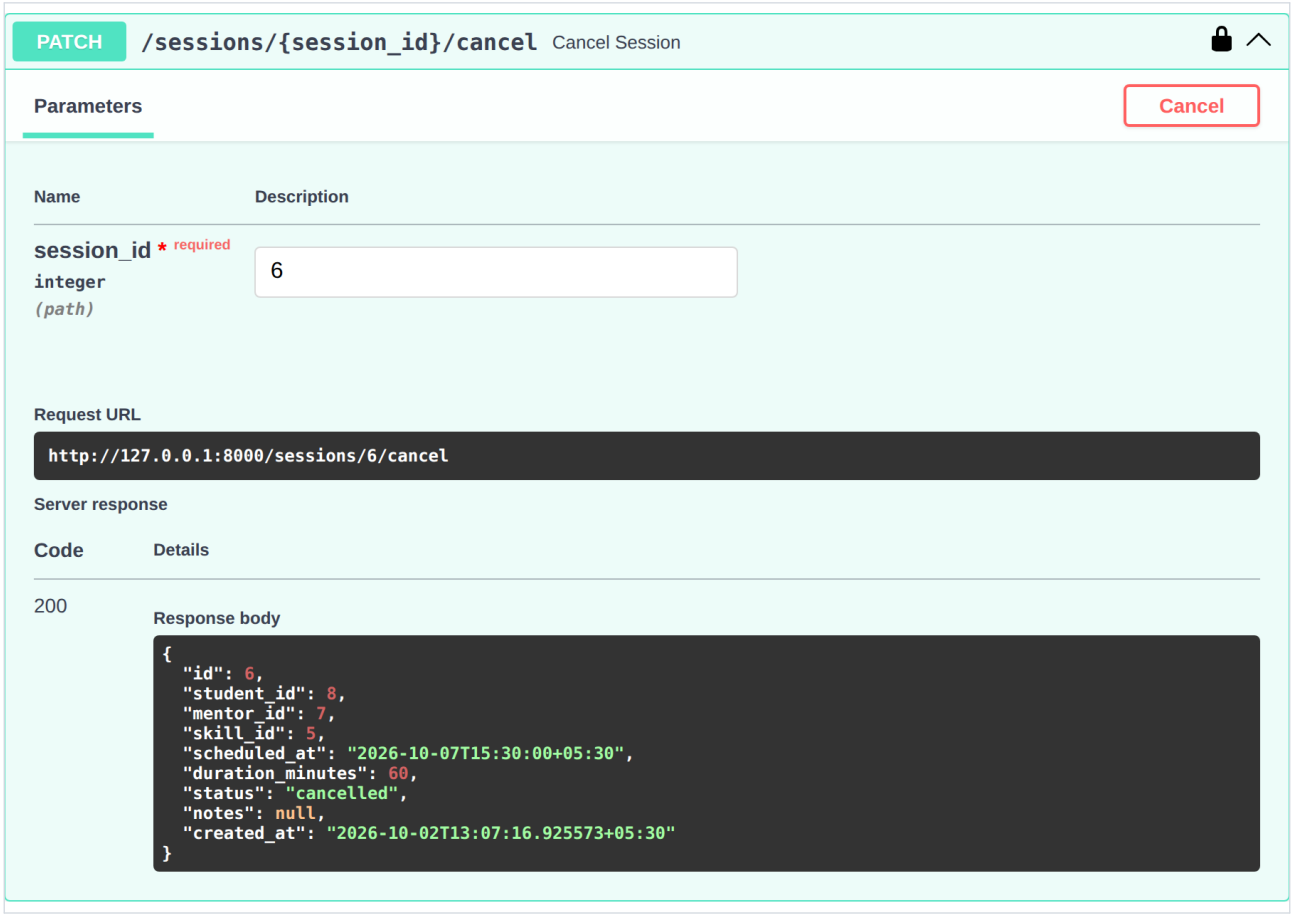


Figure 19. Cancelling a session.

STEP 18 **STUDENT** **Rahul looks at his session history (C5)**

Endpoint	`GET /sessions`	**Login needed**	Yes

With no filters Rahul sees all his sessions, newest first.

- **Notice:** both sessions are there: one `cancelled` and one `completed`.

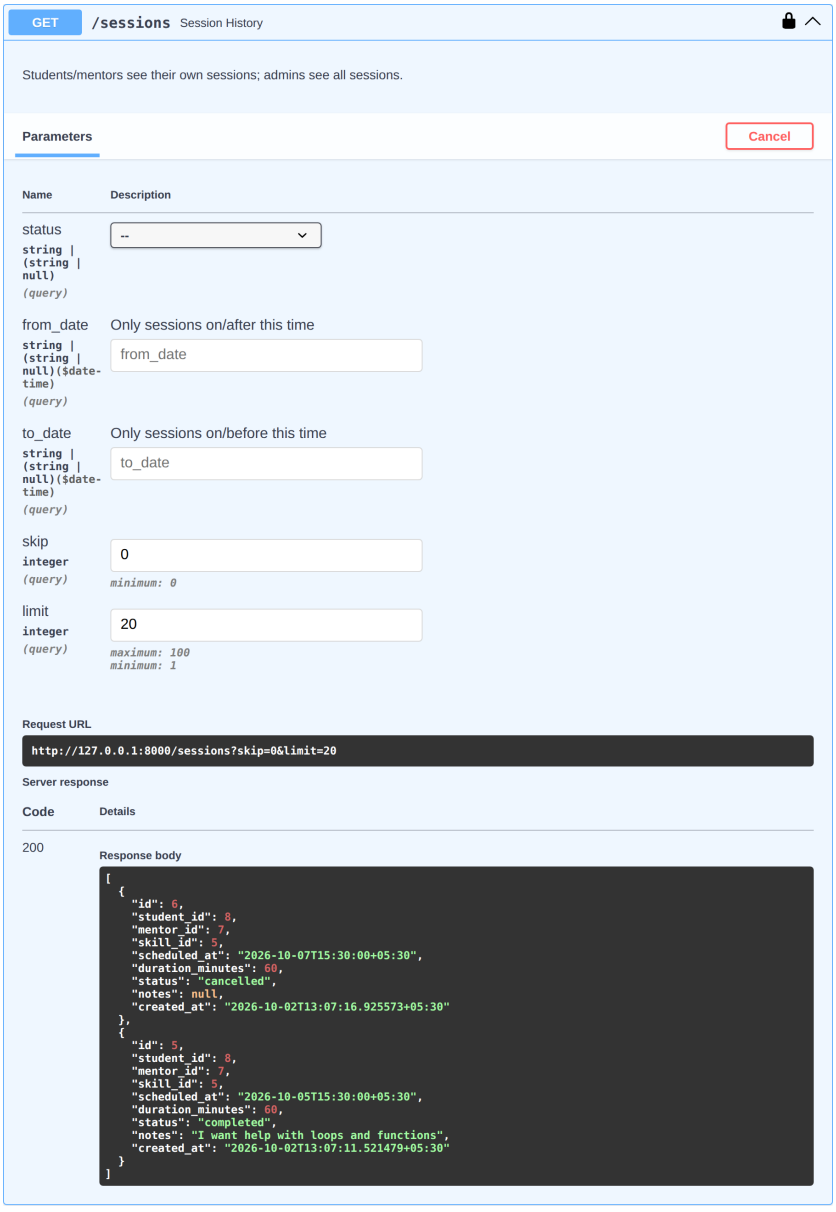


Figure 20. A student's full session history.

STEP 19 **ADMIN** **The admin lists all mentors**

Endpoint	`GET /admin/users?role=mentor`	**Login needed**	Yes – admin only

The admin logs in with the account from `.env` (`admin@skillswap.com`). Admin endpoints are blocked for everyone else (403).

- **Notice:** the admin can also filter by `is_active` and search by name or email with `q`.

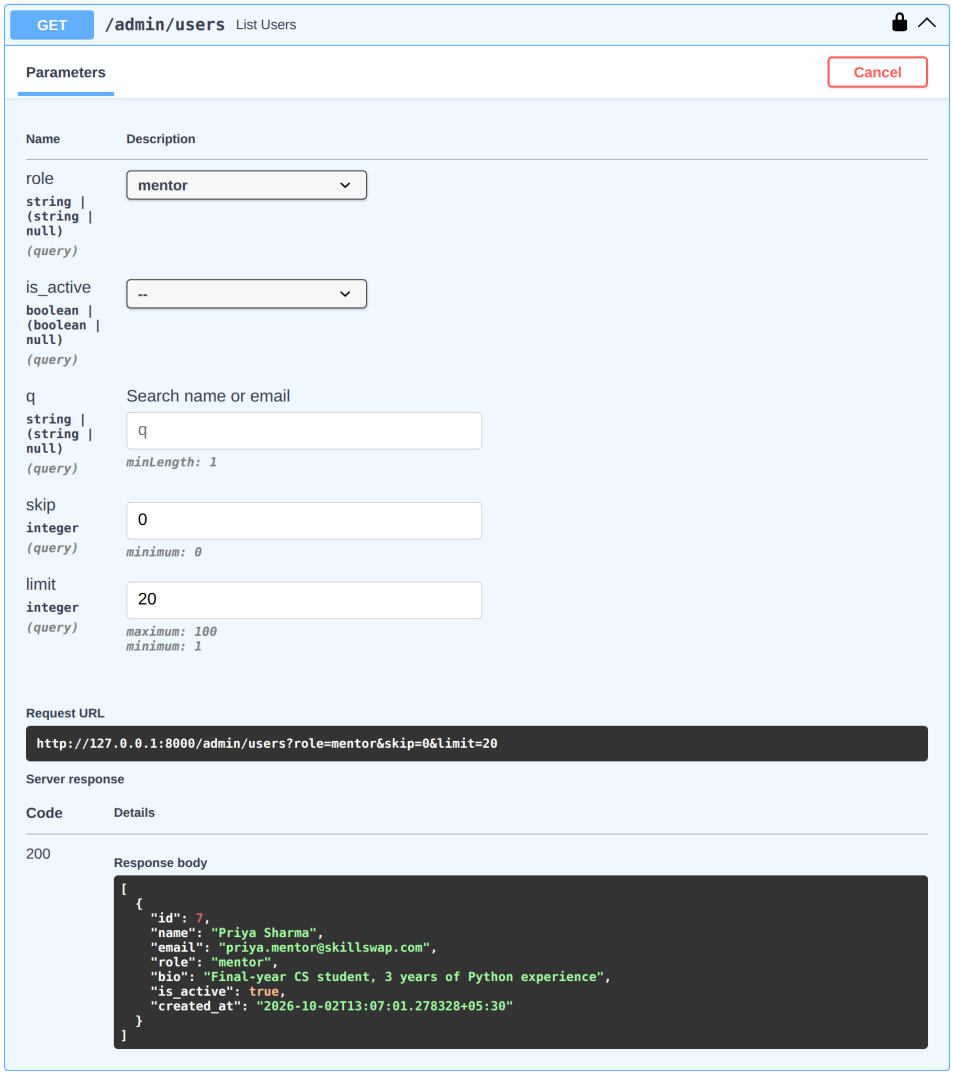


Figure 21. Admin: list users filtered by role.

STEP 20 **ADMIN** **The admin checks platform statistics**

Endpoint	`GET /admin/stats`	**Login needed**	Yes – admin only

A quick summary of the whole platform, counted in the database.

- **Notice:** users per role, number of skills, sessions per status and total reviews.

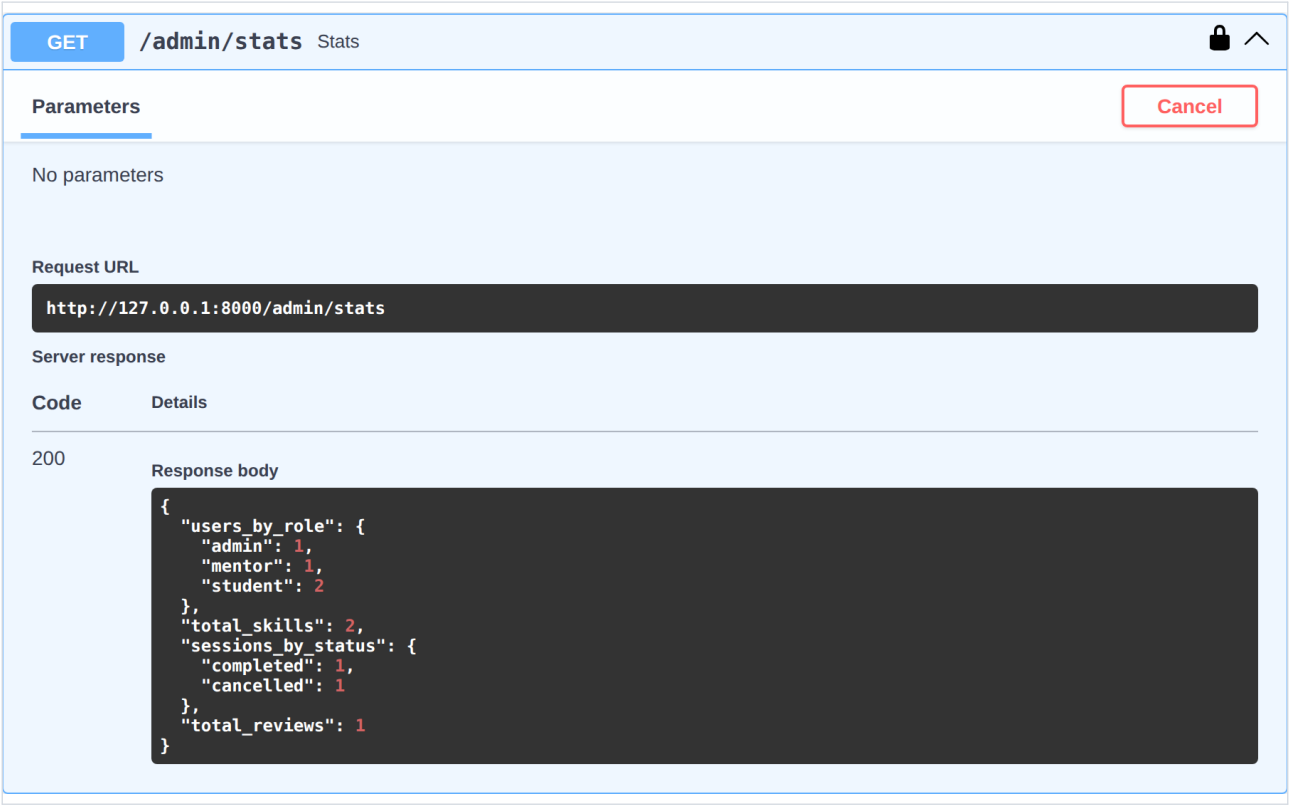


Figure 22. Admin: platform statistics.

That is the complete workflow: every core feature and every role was used, and we also saw the important error codes (401, 403, 409). Section 3 now shows the code behind each step.

SECTION 3

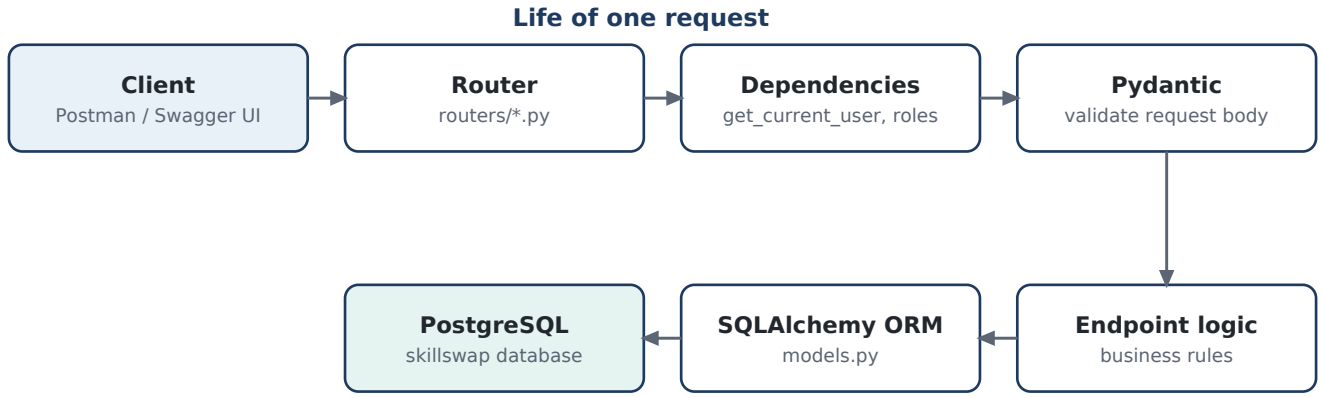
The Code, Component by Component

For every part of the problem: the code, and what each line means

3.0 How the code is organised

Think of the backend as an office. Each file has one job:

File	Job	Office example
routers/*.py	Receives each request (one file per topic) and decides what to do	the receptionist
schemas.py	Checks that the data sent is complete and correct (Pydantic)	the form checker
security.py	Passwords, tokens, “who are you?” and “are you allowed?”	the security guard
models.py	Describes the database tables	the filing cabinet labels
database.py	Opens and closes the connection to PostgreSQL	the key to the filing room
config.py + .env	Secret settings (DB password, JWT secret)	the locked drawer
main.py	Starts the app and connects all the routers	the office manager



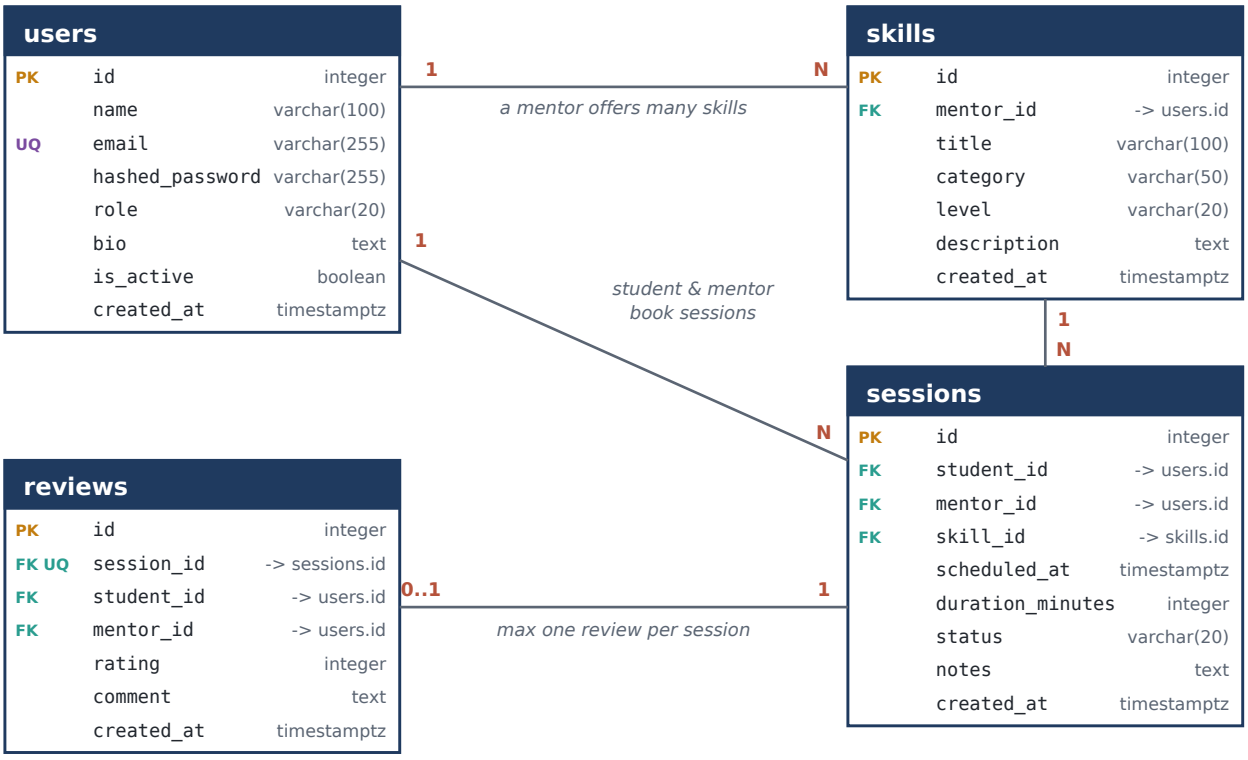
The JSON response (e.g. 201 Created) travels back the same way to the client

Figure. The path of every request through our files.

BASE

3.1 The database — where everything is stored

Problem asks	PostgreSQL database + database integration with an ORM (technical requirements 3 and 4).
In simple words	We have 4 tables: users, skills, sessions and reviews. Instead of writing SQL by hand we use **SQLAlchemy**: each Python class is a table and each attribute is a column.
Code files	`app/models.py`, `app/database.py`, `app/config.py`, `.env`
See it in Swagger	every step stores or reads rows from these tables



PK = primary key FK = foreign key (ON DELETE CASCADE) UQ = unique

Figure. The 4 tables and how they are linked (1 → N means “one to many”).

- One mentor has **many** skills. One student or mentor has **many** sessions. One skill has **many** sessions.
- One session has **at most one** review (the `session_id` column in reviews is UNIQUE).
- **ON DELETE CASCADE**: if the admin deletes a user, their skills, sessions and reviews are deleted too, so no broken rows are left.

```
app/models.py — the skills table

class Skill(Base):
    """A skill a mentor offers to teach (the mentor's skill profile)."""

    __tablename__ = "skills"
    __table_args__ = (UniqueConstraint("mentor_id", "title"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    mentor_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    title: Mapped[str] = mapped_column(String(100))
    category: Mapped[str] = mapped_column(String(50))
    level: Mapped[str] = mapped_column(String(20))
    description: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    mentor: Mapped[User] = relationship()
```

Code	What each part means
`class Skill(Base):`	A Python class that SQLAlchemy turns into a database table.
`__tablename__ = "skills"`	The real table name in PostgreSQL.
`UniqueConstraint("mentor_id", "title")`	The same mentor cannot have two skills with the same title.
`id: Mapped[int] = mapped_column(primary_key=True)`	A unique number for each row, given automatically (1, 2, 3 …).
`ForeignKey("users.id", ondelete="CASCADE")`	mentor_id must be an existing user id; deleting the user deletes the skill.
`String(100) / Text`	Text columns: up to 100 characters, or any length.
`server_default=func.now()`	PostgreSQL fills in the current time when the row is created.
`mentor: Mapped[User] = relationship()`	Lets the code write skill.mentor to get the mentor’s User object.

```
app/database.py — connecting to PostgreSQL

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False)


class Base(DeclarativeBase):
    pass


def get_db():
    """FastAPI dependency: one DB session per request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Code	What each part means
`create_engine(settings.database_url, ...)`	Connects to PostgreSQL using the URL from .env, e.g. postgresql+psycopg://postgres:postgres@localhost:5432/skillswap
`SessionLocal = sessionmaker(...)`	A factory that creates a database “session” (a conversation with the database).
`def get_db(): ... yield db ... finally: db.close()`	Every request gets its own session, and it is **always closed** afterwards, even if there was an error.

```
.env.example — secrets are kept outside the code

# PostgreSQL connection (local or Supabase)
# Supabase example: postgresql+psycopg://postgres:<password>@db.<project>.supabase.co:5432/postgres
DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/skillswap

# JWT settings
JWT_SECRET_KEY=change-me-to-a-long-random-string
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Default admin account created on startup
ADMIN_EMAIL=admin@skillswap.com
ADMIN_PASSWORD=Admin@123
```

The real `.env` is never shared (it is in `.gitignore`). `config.py` reads it, and the server refuses to start if `DATABASE_URL` or `JWT_SECRET_KEY` is missing.
When the server starts, `Base.metadata.create_all()` in `main.py` creates any missing tables, so we never had to write CREATE TABLE ourselves.

BASE

3.2 The roles: student, mentor, admin

Problem asks	“Suggested roles: Student/Mentor, Admin” and “RBAC with at least 3 roles”.
In simple words	The role is saved in the `users.role` column. People choose student or mentor when they sign up; the admin is created by the server.
Code files	`app/models.py`, `app/schemas.py`, `app/main.py`
See it in Swagger	steps 1, 7 (sign-up with a role) and 19–20 (admin)

```
app/models.py

class Role(StrEnum):
    STUDENT = "student"
    MENTOR = "mentor"
    ADMIN = "admin"
```

```
app/schemas.py — inside UserRegister

role: Literal["student", "mentor"] = "student"  # admin cannot self-register
```

```
app/main.py

def create_default_admin():
    """Admins cannot self-register, so one admin is created from .env on startup."""
    with SessionLocal() as db:
        if not db.scalar(select(User).where(User.email == settings.admin_email)):
            db.add(User(
                name="Admin",
                email=settings.admin_email,
                hashed_password=hash_password(settings.admin_password),
                role=Role.ADMIN,
            ))
            db.commit()
```

Code	What each part means
`class Role(StrEnum)`	The fixed list of allowed roles. Using a list (enum) prevents typos like “mentr”.
`Literal["student", "mentor"]`	At sign-up only these two values are accepted. Sending “admin” gives a 422 error.
`= "student"`	If no role is sent, the person becomes a student.
`if not db.scalar(select(User).where(...)):`	On startup: if the admin email from .env does not exist yet ...

Code	What each part means
`db.add(User(..., role=Role.ADMIN))`	... create the admin account with a hashed password.

E1

3.3 Authentication APIs: register and login

Problem asks	“Authentication APIs”, “JWT authentication” and “password hashing”.
In simple words	Register saves the user with a **hashed** password. Login checks the password and gives back a **JWT token** that proves who you are.
Code files	`app/routers/auth.py`, `app/security.py`
See it in Swagger	steps 1, 3, 4 and 7

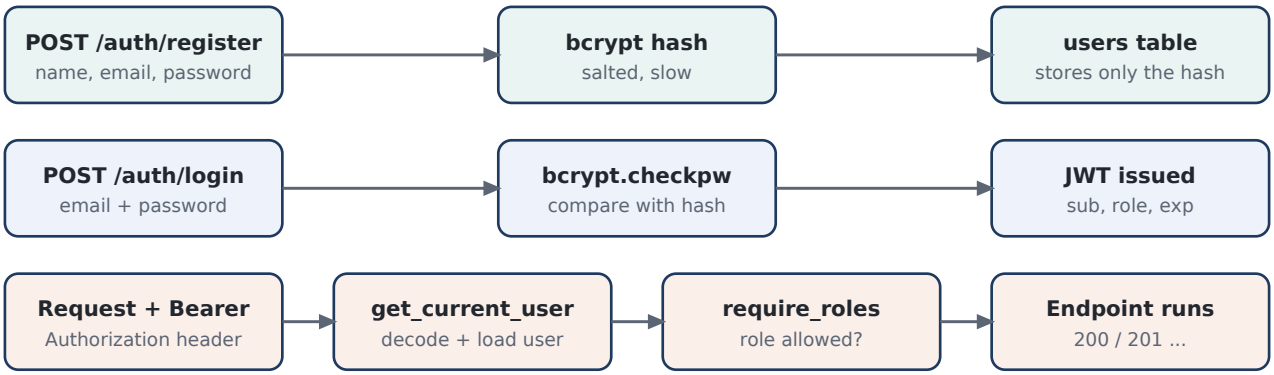


Figure. Register (top), login (middle), and using the token on every request (bottom).

```
app/routers/auth.py — register

@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
def register(data: UserRegister, db: Session = Depends(get_db)):
    if db.scalar(select(User).where(User.email == data.email)):
        raise HTTPException(status.HTTP_409_CONFLICT, "Email is already registered")
    user = User(
        name=data.name,
        email=data.email,
        hashed_password=hash_password(data.password),
        role=data.role,
        bio=data.bio,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

Code	What each part means
`@router.post("/register", ..., status_code=201)`	This function runs for POST /auth/register and answers with 201 Created.
`data: UserRegister`	FastAPI checks the JSON body with the UserRegister rules first (section 3.12). Bad data → 422.
`if db.scalar(select(User).where(User.email == data.email)):`	Look in the database: is this email already used?
`raise HTTPException(409, ...)`	If yes, stop and answer 409 Conflict.
`hashed_password=hash_password(data.password)`	Save only the bcrypt hash, never the real password.
`db.add(user); db.commit(); db.refresh(user)`	Insert the row, save it permanently, and reload it to get the new id.
`response_model=UserOut`	The answer is shaped by UserOut, which has no password field.

```
app/security.py — bcrypt

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()


def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(password.encode(), hashed.encode())
```

bcrypt turns `Priya1234` into something like `$2b$12$3/FRYcQ0U8evrZkw...`. It is one-way (it cannot be turned back), and a random **salt** makes the hash different every time.

```
app/routers/auth.py — login

@router.post("/login", response_model=Token)
def login(data: UserLogin, db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.email == data.email))
    if user is None or not verify_password(data.password, user.hashed_password):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid email or password")
    if not user.is_active:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Account is deactivated")
    return Token(access_token=create_access_token(user), role=user.role)
```

```
app/security.py — making the JWT

def create_access_token(user: User) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=settings.access_token_expire_minutes)
    payload = {"sub": str(user.id), "role": user.role, "exp": expire}
    return jwt.encode(payload, settings.jwt_secret_key, algorithm=settings.jwt_algorithm)
```

Code	What each part means
`if user is None or not verify_password(...)`	Wrong email or wrong password give the **same** 401 message, so nobody can find out which emails exist.
`if not user.is_active: ... 403`	An account blocked by the admin cannot log in.
`payload = {"sub": ..., "role": ..., "exp": ...}`	What is inside the token: the user id, the role and the expiry time (60 minutes).
`jwt.encode(payload, settings.jwt_secret_key, ...)`	Sign the token with our secret from .env. If anyone changes the token, the signature no longer matches.

E3 · E4

3.4 Protected APIs and role-based APIs

Problem asks	“Protected APIs”, “role-based APIs”, “authorization” and “protected routes using FastAPI dependencies”.
In simple words	Two small functions guard the API. `get_current_user` answers “who are you?” (else 401). `require_roles` answers “are you allowed?” (else 403).
Code files	`app/security.py` (used in every router)
See it in Swagger	steps 2 and 5 (401 vs 200), step 8 (403)

```
app/security.py — who are you?

def get_current_user(
    credentials: HTTPAuthorizationCredentials | None = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    """Dependency that reads the Bearer token and returns the logged-in user (401 otherwise)."""
    unauthorized = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or missing token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    if credentials is None:
        raise unauthorized
    try:
        payload = jwt.decode(credentials.credentials, settings.jwt_secret_key, algorithms=[settings.
    jwt_algorithm])
        user_id = int(payload["sub"])
    except (jwt.PyJWTError, KeyError, ValueError):
        raise unauthorized
    user = db.get(User, user_id)
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

Code	What each part means
`credentials = Depends(bearer_scheme)`	FastAPI reads the “Authorization: Bearer …” header for us.
`if credentials is None: raise unauthorized`	No token at all → 401 (step 2).
`jwt.decode(...)`	Check the signature and the expiry time. A fake or expired token raises an error → 401.
`user = db.get(User, user_id)`	Load the user from the database **on every request**, so a deactivated user is blocked immediately.
`return user`	The endpoint receives the logged-in user as a normal Python object.

```
app/security.py — are you allowed?

def require_roles(*roles: Role):
    """Dependency factory for role-based access control (403 if role not allowed)."""

    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "You do not have permission to perform this action")
        return user

    return checker
```

```
how the routers use it

# only mentors can create skills (step 8 shows a student getting 403)
def create_skill(data: SkillCreate, user: User = Depends(require_roles(Role.MENTOR)), ...)

# only students can book a session
def book_session(data: SessionCreate, user: User = Depends(require_roles(Role.STUDENT)), ...)
```

Code	What each part means
`def require_roles(*roles):`	We give it the allowed roles, e.g. require_roles(Role.MENTOR, Role.ADMIN).
`def checker(user = Depends(get_current_user))`	First it runs get_current_user, so the user must be logged in.
`if user.role not in roles: raise ... 403`	Logged in but the wrong role → 403 Forbidden.

Code	What each part means
`Depends(...) in the endpoint`	FastAPI runs these checks **before** the endpoint code. If a check fails, the endpoint never runs.

Ownership is the last check: a mentor may edit skills, but only **their own**:

```
app/routers/skills.py

def check_owner(skill: Skill, user: User):
    if user.role != Role.ADMIN and skill.mentor_id != user.id:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "You can only modify your own skills")
```

Action	Public	Student	Mentor	Admin
Register, login, search skills & mentors, read reviews	✔	✔	✔	✔
View / update own profile	—	✔	✔	✔
Create a skill	—	—	✔	—
Update / delete a skill	—	—	own	any
Book a session	—	✔	—	—
See sessions	—	own	own	all
Cancel a session	—	own	own	any
Complete a session	—	—	own	any
Write a review	—	own completed	—	—
Admin APIs (users, stats)	—	—	—	✔

C1 3.5 Create skill profiles

Problem asks	“Create skill profiles” + CRUD APIs.
In simple words	A mentor can create, read, update and delete the skills they teach. Two rules: no duplicate titles for the same mentor, and you cannot delete a skill that still has upcoming sessions.
Code files	`app/routers/skills.py`, `app/schemas.py` (SkillCreate, SkillUpdate)
See it in Swagger	step 6 (create) and step 8 (student blocked)

```
app/routers/skills.py — Create

@router.post("", response_model=SkillOut, status_code=status.HTTP_201_CREATED)
def create_skill(
    data: SkillCreate,
    user: User = Depends(require_roles(Role.MENTOR)),
    db: Session = Depends(get_db),
):
    check_duplicate_title(db, user.id, data.title)
    skill = Skill(mentor_id=user.id, **data.model_dump())
    db.add(skill)
    db.commit()
    db.refresh(skill)
    return skill
```

Code	What each part means
`user = Depends(require_roles(Role.MENTOR))`	Only mentors get in; the logged-in mentor is in user.
`check_duplicate_title(db, user.id, data.title)`	If this mentor already has a skill with this title → 409.

Code	What each part means
`Skill(mentor_id=user.id, **data.model_dump())`	The mentor id comes from the token, so nobody can create a skill for someone else.

app/routers/skills.py — Update

```
@router.put("/{skill_id}", response_model=SkillOut)
def update_skill(
    skill_id: int,
    data: SkillUpdate,
    user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)),
    db: Session = Depends(get_db),
):
    skill = get_skill_or_404(skill_id, db)
    check_owner(skill, user)
    changes = data.model_dump(exclude_unset=True)
    if changes.get("title"):
        check_duplicate_title(db, skill.mentor_id, changes["title"], exclude_id=skill.id)
    for field, value in changes.items():
        setattr(skill, field, value)
    db.commit()
    db.refresh(skill)
    return skill
```

Code	What each part means
`get_skill_or_404(skill_id, db)`	Find the skill by the id in the URL, or answer 404 Not Found.
`check_owner(skill, user)`	Only the owner (or an admin) may change it, otherwise 403.
`data.model_dump(exclude_unset=True)`	Only the fields that were actually sent are changed (partial update).

app/routers/skills.py — Delete

```
@router.delete("/{skill_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_skill(
    skill_id: int,
    user: User = Depends(require_roles(Role.MENTOR, Role.ADMIN)),
    db: Session = Depends(get_db),
):
    skill = get_skill_or_404(skill_id, db)
    check_owner(skill, user)
    # Business rule: cannot delete a skill that still has upcoming booked sessions
    upcoming = db.scalar(
        select(MentoringSession).where(
            MentoringSession.skill_id == skill.id,
            MentoringSession.status == SessionStatus.BOOKED,
            MentoringSession.scheduled_at > datetime.now(timezone.utc),
        )
    )
    if upcoming:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "Cancel upcoming sessions for this skill first")
    db.delete(skill)
    db.commit()
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

The delete has a **business rule**: if the skill still has a booked session in the future, we answer **400** so students are not left with a session for a skill that no longer exists. Success is **204 No Content**.

C2 · E5

3.6 Search skills and mentors

Problem asks	“Search skills/mentors” + search/filter APIs + path and query parameters.
In simple words	Every filter is optional. We start with “all skills” and add one condition for each filter the user sent.
Code files	`app/routers/skills.py` (search_skills), `app/routers/users.py` (search_mentors)
See it in Swagger	steps 9, 10 and 16

```
app/routers/skills.py

@router.get("", response_model=list[SkillOut])
def search_skills(
    q: str | None = Query(None, min_length=1, description="Search in title and description"),
    category: str | None = Query(None, min_length=1),
    level: Level | None = None,
    mentor_id: int | None = Query(None, gt=0),
    skip: int = Query(0, ge=0),
    limit: int = Query(20, ge=1, le=100),
    db: Session = Depends(get_db),
):
    stmt = select(Skill).join(Skill.mentor).where(User.is_active.is_(True))
    if q:
        stmt = stmt.where(or_(Skill.title.ilike(f"%{q}%"), Skill.description.ilike(f"%{q}%")))
    if category:
        stmt = stmt.where(Skill.category.ilike(category))
    if level:
        stmt = stmt.where(Skill.level == level)
    if mentor_id:
        stmt = stmt.where(Skill.mentor_id == mentor_id)
    return db.scalars(stmt.order_by(Skill.id).offset(skip).limit(limit)).all()
```

Code	What each part means
`q: str | None = Query(None, min_length=1)`	An optional query parameter (?q=python). If sent, it must not be empty.
`level: Level | None = None`	Must be beginner, intermediate or advanced, otherwise 422.
`limit: int = Query(20, ge=1, le=100)`	Paging: return at most 20 results by default, never more than 100.
`stmt = select(Skill).join(Skill.mentor).where(User.is_active...)`	Start with all skills of active mentors.
`if q: stmt = stmt.where(or_(title.ilike(...), description.ilike(...)))`	Keep skills whose title **or** description contains the word. ilike = ignore upper/lower case.
`.offset(skip).limit(limit)`	Skip the first N results and return the next page.

Path parameter vs query parameter: `/skills/5` → the 5 is part of the path and picks **one** skill. `/skills?q=python&level=beginner` → the part after ? **filters a list**.
Mentor search also filters by the **average rating**, which the database calculates with AVG and GROUP BY:

app/routers/users.py

```
def rating_subquery():
    return (
        select(
            Review.mentor_id,
            func.avg(Review.rating).label("avg_rating"),
            func.count(Review.id).label("review_count"),
        )
        .group_by(Review.mentor_id)
        .subquery()
    )
```

the SQL this produces

```
SELECT mentor_id, avg(rating), count(id) FROM reviews GROUP BY mentor_id;
```

E4 3.7 Admin APIs

Problem asks	Role-based APIs for the Admin role.
In simple words	All admin endpoints are in one router, and the admin check is put on the **whole router** once.
Code files	app/routers/admin.py
See it in Swagger	steps 19 and 20

app/routers/admin.py

```
router = APIRouter(prefix="/admin", tags=["Admin"], dependencies=[Depends(require_roles(Role.ADMIN))])
```

app/routers/admin.py — block or unblock a user

```
@router.patch("/users/{user_id}/status", response_model=UserOut)
def change_status(
    user_id: int,
    data: StatusUpdate,
    admin: User = Depends(require_roles(Role.ADMIN)),
    db: Session = Depends(get_db),
):
    if user_id == admin.id:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "You cannot deactivate yourself")
    user = get_user_or_404(user_id, db)
    user.is_active = data.is_active
    db.commit()
    db.refresh(user)
    return user
```

Code	What each part means
`dependencies=[Depends(require_roles(Role.ADMIN))]`	Every endpoint under /admin needs the admin role; others get 403.
`if user_id == admin.id: ... 400`	Safety rule: an admin cannot deactivate, re-role or delete themselves.
`user.is_active = data.is_active`	A blocked user cannot log in (403) and their old token stops working (401).

C3 3.8 Book mentoring sessions

Problem asks	“Book mentoring sessions” + validation and business rules.
In simple words	A student picks a skill and a time. We check: the time is in the future, the length is 30–180 minutes, the skill exists, the mentor is active, and the mentor is free at that time.
Code files	`app/schemas.py` (SessionCreate), `app/routers/sessions.py` (book_session)
See it in Swagger	steps 11 and 12

```
app/schemas.py — checking the booking data

class SessionCreate(BaseModel):
    skill_id: int = Field(gt=0)
    scheduled_at: datetime
    duration_minutes: int = Field(ge=30, le=180, description="30 to 180 minutes")
    notes: str | None = Field(default=None, max_length=500)

    @field_validator("scheduled_at")
    @classmethod
    def must_be_future(cls, v: datetime) -> datetime:
        v = as_utc(v)
        if v <= datetime.now(timezone.utc):
            raise ValueError("scheduled_at must be in the future")
        return v
```

Code	What each part means
`skill_id: int = Field(gt=0)`	Must be a whole number greater than 0.
`duration_minutes: int = Field(ge=30, le=180)`	Between 30 and 180 minutes, otherwise 422.
`@field_validator("scheduled_at")`	Our own extra check on the date …
`if v <= datetime.now(timezone.utc): raise ValueError`	… a time in the past is rejected with 422.

```
app/routers/sessions.py — booking

@router.post("", response_model=SessionOut, status_code=status.HTTP_201_CREATED)
def book_session(
    data: SessionCreate,
    user: User = Depends(require_roles(Role.STUDENT)),
    db: Session = Depends(get_db),
):
    skill = db.get(Skill, data.skill_id)
    if skill is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Skill not found")
    if not skill.mentor.is_active:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "This mentor is not available")

    # Business rule: the mentor cannot have two booked sessions that overlap
    new_start = data.scheduled_at
    new_end = new_start + timedelta(minutes=data.duration_minutes)
    booked = db.scalars(
        select(MentoringSession).where(
            MentoringSession.mentor_id == skill.mentor_id,
            MentoringSession.status == SessionStatus.BOOKED,
        )
    ).all()
    for s in booked:
        start = as_utc(s.scheduled_at)
        end = start + timedelta(minutes=s.duration_minutes)
        if start < new_end and new_start < end:
            raise HTTPException(status.HTTP_409_CONFLICT, "Mentor already has a session booked at this time")

    session = MentoringSession(
        student_id=user.id,
        mentor_id=skill.mentor_id,
        skill_id=skill.id,
        scheduled_at=data.scheduled_at,
        duration_minutes=data.duration_minutes,
        notes=data.notes,
    )
    db.add(session)
    db.commit()
    db.refresh(session)
    return session
```

Code	What each part means
`Depends(require_roles(Role.STUDENT))`	Only students can book.
`skill = db.get(Skill, data.skill_id) ... 404`	The skill must exist.
`if not skill.mentor.is_active: ... 400`	You cannot book a blocked mentor.
`booked = ... mentor_id == skill.mentor_id, status == BOOKED`	Get all the mentor’s booked sessions.
`if start < new_end and new_start < end: ... 409`	Two time ranges overlap if each one starts before the other ends. Example: 10:00–11:00 and 10:30–11:00 overlap → 409 (step 12).
`MentoringSession(student_id=user.id, mentor_id=skill.mentor_id, ...)`	Student comes from the token, mentor comes from the skill. Status starts as “booked”.

C4 3.9 Cancel sessions (and complete them)

Problem asks	“Cancel sessions”.
In simple words	Cancelling does not delete the session. It changes the status to “cancelled”, so it stays in the history.
Code files	`app/routers/sessions.py`
See it in Swagger	steps 14 (complete) and 17 (cancel)

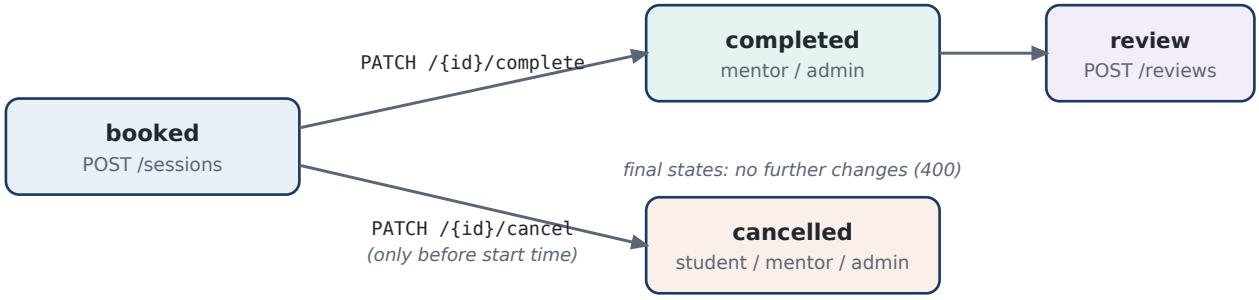


Figure. A session starts as booked and ends as completed or cancelled. Only completed sessions can be reviewed.

```
app/routers/sessions.py — shared check

def get_session_for_user(session_id: int, user: User, db: Session) -> MentoringSession:
    """Return the session if it exists (404) and the user takes part in it or is admin (403)."""
    session = db.get(MentoringSession, session_id)
    if session is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Session not found")
    if user.role != Role.ADMIN and user.id not in (session.student_id, session.mentor_id):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "This is not your session")
    return session
```

```
app/routers/sessions.py — cancel

@router.patch("/{session_id}/cancel", response_model=SessionOut)
def cancel_session(session_id: int, user: User = Depends(get_current_user), db: Session = Depends(get_db)):
    session = get_session_for_user(session_id, user, db)
    if session.status != SessionStatus.BOOKED:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, f"Cannot cancel a {session.status} session")
    if as_utc(session.scheduled_at) <= datetime.now(timezone.utc):
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "Cannot cancel a session that has already started")
    session.status = SessionStatus.CANCELLED
    db.commit()
    db.refresh(session)
    return session
```

Code	What each part means
`get_session_for_user(session_id, user, db)`	The session must exist (404) and you must be its student, its mentor, or an admin (403).
`if session.status != SessionStatus.BOOKED: ... 400`	You cannot cancel a session that is already completed or cancelled.
`if as_utc(session.scheduled_at) <= datetime.now(...): ... 400`	You cannot cancel after the start time.
`session.status = SessionStatus.CANCELLED`	Change one field and save. That is why the endpoint is PATCH (a small update), not DELETE.

Completing works the same way (`complete_session`), but only the mentor or an admin may do it.

C5 · E5

3.10 Session history

Problem asks	“Session history” + filters.
In simple words	One endpoint for everyone: students and mentors see only their own sessions, admins see all. Optional filters: status and a date range.
Code files	`app/routers/sessions.py` (session_history)
See it in Swagger	steps 13 and 18

```
app/routers/sessions.py

@router.get("", response_model=list[SessionOut])
def session_history(
    status_filter: SessionStatus | None = Query(None, alias="status"),
    from_date: datetime | None = Query(None, description="Only sessions on/after this time"),
    to_date: datetime | None = Query(None, description="Only sessions on/before this time"),
    skip: int = Query(0, ge=0),
    limit: int = Query(20, ge=1, le=100),
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
    """Students/mentors see their own sessions; admins see all sessions."""
    stmt = select(MentoringSession)
    if user.role != Role.ADMIN:
        stmt = stmt.where(or_(MentoringSession.student_id == user.id, MentoringSession.mentor_id == user.id))
    if status_filter:
        stmt = stmt.where(MentoringSession.status == status_filter)
    if from_date:
        stmt = stmt.where(MentoringSession.scheduled_at >= as_utc(from_date))
    if to_date:
        stmt = stmt.where(MentoringSession.scheduled_at <= as_utc(to_date))
    stmt = stmt.order_by(MentoringSession.scheduled_at.desc()).offset(skip).limit(limit)
    return db.scalars(stmt).all()
```

Code	What each part means
`status_filter: ... = Query(None, alias="status")`	The URL uses ?status=booked. The name in Python is different because status is already used.
`if user.role != Role.ADMIN: stmt = stmt.where(or_(...))`	Non-admins only get rows where they are the student **or** the mentor.
`if from_date: ... >= ; if to_date: ... <=`	Optional date range.
`.order_by(MentoringSession.scheduled_at.desc())`	Newest first.

C6 3.11 Reviews and ratings

Problem asks	“Reviews and ratings”.
In simple words	After a completed session the student gives 1–5 stars and a comment. Each session can be reviewed only once. A mentor’s average rating is calculated from all their reviews.
Code files	`app/routers/reviews.py`, `app/routers/users.py` (average rating)
See it in Swagger	steps 15 and 16

```
app/routers/reviews.py

@router.post("", response_model=ReviewOut, status_code=status.HTTP_201_CREATED)
def create_review(
    data: ReviewCreate,
    user: User = Depends(require_roles(Role.STUDENT)),
    db: Session = Depends(get_db),
):
    session = db.get(MentoringSession, data.session_id)
    if session is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Session not found")
    if session.student_id != user.id:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "You can only review your own sessions")
    if session.status != SessionStatus.COMPLETED:
        raise HTTPException(status.HTTP_400_BAD_REQUEST, "You can only review completed sessions")
    if db.scalar(select(Review).where(Review.session_id == session.id)):
        raise HTTPException(status.HTTP_409_CONFLICT, "This session has already been reviewed")

    review = Review(
        session_id=session.id,
        student_id=user.id,
        mentor_id=session.mentor_id,
        rating=data.rating,
        comment=data.comment,
    )
    db.add(review)
    db.commit()
    db.refresh(review)
    return review
```

Code	What each part means
`rating: int = Field(ge=1, le=5)` (in ReviewCreate)	The rating must be 1, 2, 3, 4 or 5, otherwise 422.
`if session is None: ... 404`	The session must exist.
`if session.student_id != user.id: ... 403`	You can only review your own session.
`if session.status != SessionStatus.COMPLETED: ... 400`	Only completed sessions can be reviewed (a cancelled one gives 400).
`if db.scalar(select(Review).where(...)): ... 409`	Already reviewed → 409. The database also has a UNIQUE constraint as a second safety net.
`mentor_id=session.mentor_id`	Saved on the review so we can quickly calculate each mentor’s average.

Reviews also have Read (`GET /reviews?mentor_id=&min_rating=`), Update (`PUT /reviews/{id}`, author only) and Delete (`DELETE /reviews/{id}`, author or admin), so this is full CRUD.

E6 3.12 Validation, business rules and errors

Problem asks	“Validation/business-rule handling”, Pydantic validation, HTTP status codes and exception handling.
In simple words	Two layers. **Pydantic** checks the shape of the data before our code runs (422). Our endpoint code then checks the **business rules** and answers with a clear status code.
Code files	`app/schemas.py`, all routers, `app/main.py`
See it in Swagger	steps 2, 8, 12 and every success step

```
app/schemas.py — sign-up rules

class UserRegister(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: EmailStr
    password: str = Field(min_length=8, max_length=64)
    role: Literal["student", "mentor"] = "student"  # admin cannot self-register
    bio: str | None = Field(default=None, max_length=500)

    @field_validator("password")
    @classmethod
    def strong_password(cls, v: str) -> str:
        if not any(c.isalpha() for c in v) or not any(c.isdigit() for c in v):
            raise ValueError("password must contain at least one letter and one digit")
        if len(v.encode()) > 72:
            raise ValueError("password is too long")
        return v
```

Code	What each part means
`name: str = Field(min_length=2, max_length=100)`	Name length 2–100.
`email: EmailStr`	Must look like a real email (has @ and a domain).
`password: str = Field(min_length=8, max_length=64)`	Password length 8–64.
`def strong_password(...)`	Our own rule: at least one letter and one digit.

If several things are wrong, Pydantic lists **all** of them at once:

```
real response (shortened)

POST /auth/register   {"name": "X", "email": "bad", "password": "short", "role": "admin"}
422 Unprocessable Content
  body.name      String should have at least 2 characters
  body.email     value is not a valid email address: An email address must have an @-sign.
  body.password  String should have at least 8 characters
  body.role      Input should be 'student' or 'mentor'
```

Status codes we use

Code	Meaning	Example from our workflow
200 OK	It worked	login, search, cancel, complete (steps 3, 9, 14, 17)
201 Created	A new row was created	register, create skill, book, review (steps 1, 6, 11, 15)
204 No Content	Deleted	DELETE /skills/{id}, /reviews/{id}
400 Bad Request	Data is fine, but a rule says no	cancel a completed session; review a cancelled one
401 Unauthorized	Who are you? (no or bad token)	step 2
403 Forbidden	You are not allowed	step 8
404 Not Found	That id does not exist	GET /skills/999999

Code	Meaning	Example from our workflow
409 Conflict	Duplicate or clash	step 12, email already used, second review
422 Unprocessable	Data is wrong (Pydantic)	rating 6, date in the past, bad email

Exception handling

- Expected problems use `raise HTTPException(status_code, detail)`, which FastAPI turns into JSON like `{"detail": "Skill not found"}`.
- Small helpers like `get_skill_or_404` keep the 404 handling the same everywhere.
- A global handler catches a database duplicate error (`IntegrityError`) and answers 409 instead of crashing with 500:

```
app/main.py

@app.exception_handler(IntegrityError)
async def integrity_error_handler(request: Request, exc: IntegrityError):
    """Safety net for duplicate records that slip past the explicit checks."""
    return JSONResponse(status_code=status.HTTP_409_CONFLICT, content={"detail": "Duplicate or conflicting
    record"})
```

TEST

3.13 Testing: Swagger and Postman

Problem asks	Swagger/OpenAPI documentation and a Postman collection (technical requirements 15 and 16).
In simple words	Swagger is the live documentation we used in section 2. Postman runs **53 saved requests** automatically and checks every status code.
Code files	`postman/SkillSwap.postman_collection.json`
See it in Swagger	all of section 2

Postman folder	Requests	What it checks
1. Auth	9	register, duplicate email 409, bad data 422, login, wrong password 401
2. Users & Mentors	3	my profile, no token 401, update profile
3. Skills	11	create, duplicate 409, student 403, bad level 422, search, 404, update
4. Sessions	12	book, clash 409, past date 422, mentor can’t book 403, history, cancel, cancel twice 400, complete
5. Reviews	7	review cancelled 400, create, duplicate 409, rating 6 → 422, list, update
6. Admin	8	list users, student 403, stats, deactivate → login 403, change role, 404
7. Cleanup	3	delete review, skill and user (204)

- Tokens and ids are saved automatically by small scripts (for example `pm.collectionVariables.set("student_token", ...)`), so the requests run one after another like a real user.
- To run: Postman → Import the file → right-click the collection → **Run collection** → Run.
- Result on our PostgreSQL database: **53 requests, 53 tests passed, 0 failed**.

3.14 The 16 technical requirements

Section 4 goes through all 16 mandatory requirements one by one: what was expected, how we implemented it, the code and the proof.

SECTION 4

The 16 Technical Requirements

Each mandatory requirement: what was expected, how we built it, the code, and the proof

This section answers the question "**how did you implement requirement X?**". Every requirement has the same four parts: the **expected** implementation (copied from the requirement sheet), **how we implemented it**, the **code**, and the **proof** (a Swagger step from section 2, a Postman test, or real output).

#	Requirement	Status	Presented by
1	FastAPI REST API	✔ Done	Member 1
2	CRUD APIs	✔ Done	Member 1
3	PostgreSQL Database	✔ Done	Member 3
4	Database Integration	✔ Done	Member 3
5	Pydantic Validation	✔ Done	Member 5
6	JWT Authentication	✔ Done	Member 4
7	Password Hashing	✔ Done	Member 4
8	Authorization	✔ Done	Member 4
9	RBAC (3 roles)	✔ Done	Member 4
10	Protected Routes	✔ Done	Member 4
11	Path & Query Parameters	✔ Done	Member 2
12	HTTP Status Codes	✔ Done	Member 5
13	Exception Handling	✔ Done	Member 5
14	Environment Variables	✔ Done	Member 3
15	Swagger / OpenAPI	✔ Done	Member 2
16	Postman Testing	✔ Done	Member 5

REQUIREMENT 1

FastAPI REST API

Presented by **Member 1**

Expected: *Build the backend using FastAPI with proper API routes.*

How we implemented it

- The whole backend is a FastAPI app created in `app/main.py`.
- Routes are split into **6 routers** (`APIRouter`), one per topic, each with its own URL prefix and Swagger tag.
- **29 endpoints** in total, following REST: the URL names the resource (`/skills`), the HTTP method names the action (POST create, GET read, PUT/PATCH update, DELETE delete).
- Every endpoint declares its response shape with `response_model` and its success code with `status_code`.
- The app runs on the Uvicorn server: `uvicorn app.main:app --reload`.

Router file	URL prefix	Swagger tag	Endpoints
routers/auth.py	/auth	Authentication	2
routers/users.py	/users	Users & Mentors	4
routers/skills.py	/skills	Skills	5
routers/sessions.py	/sessions	Mentoring Sessions	5
routers/reviews.py	/reviews	Reviews & Ratings	5
routers/admin.py	/admin	Admin	7
main.py	/	Health	1
		Total	**29**

how routes are built

```
# app/routers/skills.py — one router per topic
router = APIRouter(prefix="/skills", tags=["Skills"])

@router.get("/{skill_id}", response_model=SkillOut)        # GET /skills/5
def get_skill(skill_id: int, db: Session = Depends(get_db)):
    return get_skill_or_404(skill_id, db)

# app/main.py — the app plugs every router in
app = FastAPI(title="SkillSwap API", description="...", version="1.0.0", lifespan=lifespan)
app.include_router(auth.router)
app.include_router(users.router)
app.include_router(skills.router)
app.include_router(sessions.router)
app.include_router(reviews.router)
app.include_router(admin.router)
```

✔ **Proof:** all 29 routes are listed in Swagger (section 2.1, Figure 2) and all are called in section 2.

REQUIREMENT 2

CRUD APIs

Presented by **Member 1**

Expected: *Implement Create, Read, Update and Delete operations for the main entities.*

How we implemented it

- Our main entities are **User, Skill, Session and Review**. Each one has Create, Read, Update and Delete.
- Create returns **201 Created**, Read and Update return **200 OK**, Delete returns **204 No Content**.
- Sessions are “updated” by changing their status (cancel / complete) so the history is kept; the admin can still delete them.

Entity	Create	Read	Update	Delete
User	POST /auth/register	GET /users/me GET /admin/users GET /admin/users/{id}	PUT /users/me PATCH /admin/users/{id}/role PATCH .../{id}/status	DELETE /admin/users/{id}
Skill	POST /skills	GET /skills GET /skills/{id}	PUT /skills/{id}	DELETE /skills/{id}
Session	POST /sessions	GET /sessions GET /sessions/{id}	PATCH /sessions/{id}/cancel PATCH /sessions/{id}/complete	DELETE /admin/sessions/{id}
Review	POST /reviews	GET /reviews GET /reviews/{id}	PUT /reviews/{id}	DELETE /reviews/{id}

```
app/routers/skills.py — the full CRUD set for skills

@router.post("", response_model=SkillOut, status_code=status.HTTP_201_CREATED)   # Create
@router.get("", response_model=list[SkillOut])                                   # Read (list + search)
@router.get("/{skill_id}", response_model=SkillOut)                              # Read (one)
@router.put("/{skill_id}", response_model=SkillOut)                              # Update
@router.delete("/{skill_id}", status_code=status.HTTP_204_NO_CONTENT)            # Delete
```

✔ **Proof:** Create: steps 1, 6, 11, 15. Read: steps 5, 9, 13, 16, 18. Update: steps 14, 17. Delete: Postman folder “7. Cleanup” (delete review, skill and user → 204).

REQUIREMENT 3

PostgreSQL Database

Presented by **Member 3**

Expected: *Store application data in PostgreSQL. Supabase PostgreSQL may be used.*

How we implemented it

- All data is stored in a PostgreSQL 18 database called **skillswap** on our machine, managed with pgAdmin 4.
- The connection uses the **psycopg 3** driver. The address is one line in `.env`.
- To use **Supabase** instead, we only change that one line to the Supabase connection string. No code changes.
- PostgreSQL gives us real foreign keys, unique constraints, cascade delete and time-zone aware dates.

```
the only setting needed

# .env — local PostgreSQL
DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/skillswap

# .env — Supabase (same code, only the URL changes)
DATABASE_URL=postgresql+psycopg://postgres:<password>@db.<project>.supabase.co:5432/postgres
```

```
real output from our PostgreSQL database

skillswap=# \dt
            List of tables
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | reviews  | table | postgres
 public | sessions | table | postgres
 public | skills   | table | postgres
 public | users    | table | postgres
(4 rows)
```

✔ **Proof:** the 4 tables exist in PostgreSQL (output above); in pgAdmin open skillswap → Schemas → public → Tables → View Data to see Priya, Rahul, their skills, sessions and the review from section 2.

REQUIREMENT 4

Database Integration

Presented by **Member 3**

Expected: *Connect FastAPI to PostgreSQL and implement database operations using an appropriate ORM/database layer.*

How we implemented it

- We use **SQLAlchemy 2** as the ORM. Each table is a Python class in `models.py` (section 3.1).
- `database.py` creates the **engine** (connection pool) and `get_db()`, a FastAPI dependency that gives each request its own database session and always closes it.
- Tables are created automatically on startup with `Base.metadata.create_all()`.
- All queries go through the ORM, so values are always sent as parameters — this protects against **SQL injection**.

```
app/database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.config import settings

engine = create_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False)


class Base(DeclarativeBase):
    pass


def get_db():
    """FastAPI dependency: one DB session per request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Operation	ORM code in our project	SQL that SQLAlchemy runs
Insert	db.add(skill); db.commit()	INSERT INTO skills (...) VALUES (...)

Operation	ORM code in our project	SQL that SQLAlchemy runs
Find by id	db.get(Skill, skill_id)	SELECT ... FROM skills WHERE id = 5
Filter	select(User).where(User.email == data.email)	SELECT ... FROM users WHERE email = ...
Search	Skill.title.ilike(f"%{q}%")	WHERE title ILIKE '%python%'
Update	setattr(skill, field, value); db.commit()	UPDATE skills SET ... WHERE id = 5
Delete	db.delete(skill); db.commit()	DELETE FROM skills WHERE id = 5
Aggregate	func.avg(Review.rating) ... group_by(Review.mentor_id)	SELECT avg(rating) ... GROUP BY mentor_id

```
how an endpoint receives and uses the database

def create_skill(data: SkillCreate, user: User = Depends(require_roles(Role.MENTOR)),
                 db: Session = Depends(get_db)):          # <- FastAPI gives us a DB session
    ...
    db.add(skill)
    db.commit()
    db.refresh(skill)
```

✔ **Proof:** every successful step in section 2 reads or writes PostgreSQL through these ORM calls.

REQUIREMENT 5

Pydantic Validation

Presented by **Member 5**

Expected: *Validate request data, required fields, data types, ranges, formats and business rules.*

How we implemented it

- Every request body is a Pydantic class in `schemas.py`. FastAPI checks it **before** our code runs.
- Query parameters are validated the same way with `Query(...)` (for example `limit` must be 1–100).
- If anything is wrong the client gets **422** with a list of **every** problem, field by field.
- Response models (`UserOut`, `SkillOut` ...) control what goes out, e.g. the password hash is never returned.

What is checked	Example in our code	Bad input	Real error message
Required fields	name: str (no default)	name missing	Field required
Data types	skill_id: int	"abc"	Input should be a valid integer
Ranges	duration_minutes: Field(ge=30, le=180) rating: Field(ge=1, le=5)	500	Input should be less than or equal to 180
Lengths	name: Field(min_length=2, max_length=100)	"X"	String should have at least 2 characters
Formats	email: EmailStr scheduled_at: datetime	"bad" "tomorrow"	value is not a valid email address Input should be a valid datetime
Allowed values	role: Literal["student", "mentor"] level: Level (enum)	"admin"	Input should be 'student' or 'mentor'
Business rules	@field_validator: future date, password has a letter and a digit	2020-01-01	scheduled_at must be in the future

```
app/schemas.py — types, ranges, format and a business rule in one class

class SessionCreate(BaseModel):
    skill_id: int = Field(gt=0)
    scheduled_at: datetime
    duration_minutes: int = Field(ge=30, le=180, description="30 to 180 minutes")
    notes: str | None = Field(default=None, max_length=500)

    @field_validator("scheduled_at")
    @classmethod
    def must_be_future(cls, v: datetime) -> datetime:
        v = as_utc(v)
        if v <= datetime.now(timezone.utc):
            raise ValueError("scheduled_at must be in the future")
        return v
```

```
real response from our API

POST /sessions   {"skill_id": "abc", "scheduled_at": "tomorrow", "duration_minutes": 500}
422 Unprocessable Content
  body.skill_id          Input should be a valid integer, unable to parse string as an integer
  body.scheduled_at      Input should be a valid datetime or date, input is too short
  body.duration_minutes  Input should be less than or equal to 180
```

✔ **Proof:** Postman tests “Register invalid data (422)”, “Create skill invalid level (422)”, “Book in the past (422)”, “Invalid rating (422)” all pass.

REQUIREMENT 6

JWT Authentication

Presented by **Member 4**

Expected: *Implement registration/login and issue JWT access tokens.*

How we implemented it

- `POST /auth/register` creates a student or mentor account (201).
- `POST /auth/login` checks email and password and returns a **JWT access token** (library: PyJWT).
- The token contains the user id (`sub`), the role and the expiry time (`exp`, 60 minutes). It is signed with HS256 and the secret `JWT_SECRET_KEY` from .env.
- The client sends it on every request as `Authorization: Bearer <token>`; the server verifies the signature and expiry.

```
app/security.py — issuing the token

def create_access_token(user: User) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=settings.access_token_expire_minutes)
    payload = {"sub": str(user.id), "role": user.role, "exp": expire}
    return jwt.encode(payload, settings.jwt_secret_key, algorithm=settings.jwt_algorithm)
```

```
app/routers/auth.py — the login endpoint

@router.post("/login", response_model=Token)
def login(data: UserLogin, db: Session = Depends(get_db)):
    user = db.scalar(select(User).where(User.email == data.email))
    if user is None or not verify_password(data.password, user.hashed_password):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid email or password")
    if not user.is_active:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Account is deactivated")
    return Token(access_token=create_access_token(user), role=user.role)
```

```
a real login response

{"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
    eyJzdWIiOiIxIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwOTI4ODQ2fQ.wmDY...",
 "token_type": "bearer", "role": "admin"}

decoded payload:  {"sub": "1", "role": "admin", "exp": 1790928846}
```

✔ **Proof:** steps 1 and 7 (register), step 3 (login returns the token), step 4 (token used in Swagger). Postman “Login admin / mentor / student” save the tokens; “Login wrong password (401)” passes.

REQUIREMENT 7

Password Hashing

Presented by **Member 4**

Expected: *Never store plain-text passwords. Store securely hashed passwords.*

How we implemented it

- Passwords are hashed with **bcrypt** (cost factor 12, random salt) before they are saved.
- At login we use `bcrypt.checkpw` to compare; the real password is never stored or logged.
- The `UserOut` response model has no password field, so even the hash is never sent to clients.
- The admin password from .env is hashed the same way when the admin account is created.

```
app/security.py

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()


def verify_password(password: str, hashed: str) -> bool:
    return bcrypt.checkpw(password.encode(), hashed.encode())
```

```
real rows in PostgreSQL (shortened)

skillswap=# SELECT id, email, role, hashed_password FROM users;
 id |            email            |  role   |         hashed_password
----+-----------------------------+---------+---------------------------------
  1 | admin@skillswap.com         | admin   | $2b$12$3/FRYcQ0U8evrZkwhKGOpu...
  7 | priya.mentor@skillswap.com  | mentor  | $2b$12$IPrrFhWpDHzO.hh3Afho5e...
  8 | rahul.student@skillswap.com | student | $2b$12$zjub0yOCRlN3pvWvuVH6t....
```

✔ **Proof:** the database only contains bcrypt hashes (above). Priya’s password “Priya1234” appears nowhere in the database.

REQUIREMENT 8

Authorization

Presented by **Member 4**

Expected: *Protect APIs based on the authenticated user's permissions.*

How we implemented it

- Being logged in is not enough: the API also checks **what you own**.
- Mentors can only update/delete **their own** skills; only the student and mentor of a session (or an admin) can see or cancel it; only the author can edit a review.
- Sensitive ids are taken from the token, not from the request body (e.g. `mentor_id=user.id` when creating a skill), so you cannot act on behalf of someone else.
- Session history automatically shows only your own sessions unless you are an admin.

Rule	Function	If broken
Edit / delete a skill: owner or admin	check_owner()	403 You can only modify your own skills
See / cancel / complete a session: participant or admin	get_session_for_user()	403 This is not your session
Review a session: its own student	create_review()	403 You can only review your own sessions
Edit a review: its author	update_review()	403 You can only edit your own reviews
Delete a review: author or admin	delete_review()	403 You can only delete your own reviews
Session history: own sessions (admin: all)	session_history()	other people's sessions are filtered out

app/routers/sessions.py

```
def get_session_for_user(session_id: int, user: User, db: Session) -> MentoringSession:
    """Return the session if it exists (404) and the user takes part in it or is admin (403)."""
    session = db.get(MentoringSession, session_id)
    if session is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Session not found")
    if user.role != Role.ADMIN and user.id not in (session.student_id, session.mentor_id):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "This is not your session")
    return session
```

✔ **Proof:** step 13 (Priya only sees her own sessions); Postman "Complete session as student (403)" passes.

REQUIREMENT 9

RBAC — Role-Based Access Control

Presented by **Member 4**

Expected: *Implement at least 3 roles where applicable, such as User/Student, Staff/Organizer and Admin.*

How we implemented it

- We have exactly the 3 roles the problem needs: **student** (user), **mentor** (staff/organizer of sessions) and **admin**.
- The role is stored in `users.role`; students and mentors choose it at sign-up, the admin is created from .env (nobody can sign up as admin).
- A reusable dependency `require_roles(...)` allows only the listed roles and answers **403** to everyone else.
- The admin router is protected as a whole with one line.

app/models.py

```
class Role(StrEnum):
    STUDENT = "student"
    MENTOR = "mentor"
    ADMIN = "admin"
```

app/security.py

```
def require_roles(*roles: Role):
    """Dependency factory for role-based access control (403 if role not allowed)."""

    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "You do not have permission to perform this action")
        return user

    return checker
```

Endpoint	Allowed roles
POST /skills	mentor
PUT / DELETE /skills/{id}	mentor (own), admin
POST /sessions	student
PATCH /sessions/{id}/complete	mentor (own), admin
POST /reviews	student
everything under /admin	admin

app/routers/admin.py — one line protects all 7 admin endpoints

```
router = APIRouter(prefix="/admin", tags=["Admin"], dependencies=[Depends(require_roles(Role.ADMIN))])
```

✔ **Proof:** step 8 (student → 403 on POST /skills), steps 19–20 (admin). Postman “Create skill as student (403)”, “Book as mentor (403)”, “List users as student (403)” pass.

REQUIREMENT 10

Protected Routes

Presented by **Member 4**

Expected: *Use FastAPI dependencies to identify the current user and protect private endpoints.*

How we implemented it

- `get_current_user` is a FastAPI **dependency**: it reads the Bearer token, verifies it, loads the user from the database and returns it.
- An endpoint becomes private just by adding `user: User = Depends(get_current_user)` (or `require_roles(...)`, which uses it).
- **20 of the 29 endpoints are protected.** The 9 public ones are: register, login, search skills, get skill, search mentors, mentor profile, list reviews, get review, health.
- Missing, fake or expired token → **401**. A deactivated user is also rejected because we re-load the user on every request.

app/security.py

```
def get_current_user(
    credentials: HTTPAuthorizationCredentials | None = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    """Dependency that reads the Bearer token and returns the logged-in user (401 otherwise)."""
    unauthorized = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or missing token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    if credentials is None:
        raise unauthorized
    try:
        payload = jwt.decode(credentials.credentials, settings.jwt_secret_key, algorithms=[settings.
    jwt_algorithm])
        user_id = int(payload["sub"])
    except (jwt.PyJWTError, KeyError, ValueError):
        raise unauthorized
    user = db.get(User, user_id)
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

app/routers/users.py — a protected route

```
@router.get("/me", response_model=UserOut)
def get_me(user: User = Depends(get_current_user)):    # FastAPI runs get_current_user first
    return user
```

✔ **Proof:** step 2 (no token → 401) and step 5 (same request with token → 200). In Swagger the 20 protected routes show a lock icon. Postman “Get my profile without token (401)” passes.

REQUIREMENT 11

Path & Query Parameters

Presented by **Member 2**

Expected: *Use path parameters and query parameters for resource lookup, searching and filtering.*

How we implemented it

- **Path parameters** pick one resource: `/skills/{skill_id}`, `/sessions/{session_id}`, `/reviews/{review_id}`, `/users/mentors/{mentor_id}`, `/admin/users/{user_id}`. FastAPI converts them to int; unknown ids → 404.
- **Query parameters** search and filter lists. All are optional and validated (wrong values → 422).
- Every list endpoint has paging with `skip` and `limit` (max 100).

Endpoint	Query parameters	Used for
GET /skills	q, category, level, mentor_id, skip, limit	search skills by text, category, level, mentor
GET /users/mentors	skill, category, min_rating, skip, limit	find mentors by skill and rating
GET /sessions	status, from_date, to_date, skip, limit	filter session history
GET /reviews	mentor_id, min_rating, skip, limit	reviews for a mentor
GET /admin/users	role, is_active, q, skip, limit	admin user search

```
query and path parameters in our code

@router.get("", response_model=list[SkillOut])
def search_skills(
    q: str | None = Query(None, min_length=1, description="Search in title and description"),   # ?q=python
    category: str | None = Query(None, min_length=1),                                           #
    ?category=Programming
    level: Level | None = None,                                                                 # ?level=beginner
    mentor_id: int | None = Query(None, gt=0),
    skip: int = Query(0, ge=0),
    limit: int = Query(20, ge=1, le=100),
    ...

@router.patch("/{session_id}/cancel", response_model=SessionOut)                               # path parameter
def cancel_session(session_id: int, ...):
```

✔ **Proof:** steps 9 (`?q=python&level=beginner`), 10, 13 (`?status=booked`), 19 (`?role=mentor`) use query parameters; steps 14, 16, 17 use path parameters (`/sessions/5/complete`, `/users/mentors/7`).

REQUIREMENT 12

HTTP Status Codes

Presented by **Member 5**

Expected: *Use appropriate status codes such as 200, 201, 400, 401, 403, 404 and 409.*

How we implemented it

- We use all the required codes, plus **204** for deletes and **422** for invalid data.
- Success codes are set on the route (`status_code=status.HTTP_201_CREATED`); error codes are raised with `HTTPException`.
- We use the named constants from `fastapi.status` so the code is easy to read.

Code	When we use it	Proof
200 OK	successful read / update / login	steps 3, 5, 9, 14, 17
201 Created	register, create skill, book session, create review	steps 1, 6, 11, 15
204 No Content	successful delete	Postman “7. Cleanup”
400 Bad Request	business rule broken (cancel a cancelled session, review a cancelled one, …)	Postman “Cancel already-cancelled session”
401 Unauthorized	no / invalid / expired token, wrong password	step 2
403 Forbidden	wrong role, not the owner, deactivated account at login	step 8
404 Not Found	id does not exist	Postman “Get missing skill (404)”
409 Conflict	email already used, duplicate skill title, double booking, second review	step 12
422 Unprocessable	request data fails Pydantic validation	requirement 5

```
app/routers/auth.py

@router.post("/register", response_model=UserOut, status_code=status.HTTP_201_CREATED)
...
    raise HTTPException(status.HTTP_409_CONFLICT, "Email is already registered")
...
    raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid email or password")
```

✔ **Proof:** every Postman request checks its exact expected status code (53 of 53 pass).

REQUIREMENT 13

Exception Handling

Presented by **Member 5**

Expected: *Handle invalid input, missing resources, duplicate records and business-rule failures.*

How we implemented it

- Every problem returns a clean JSON error `{"detail": "..."}` with the right status code — the server never crashes with a 500 for these cases.
- **Invalid input** is handled automatically by Pydantic (422).
- **Missing resources** use small helpers (`get_skill_or_404`, `get_review_or_404`, `get_user_or_404`).
- **Duplicate records** are checked before saving (409). As a safety net, a global handler turns a database `IntegrityError` into 409 too.
- **Business-rule failures** raise 400 or 409 with a message that explains the rule.

Required case	How we handle it	Real message from our API
Invalid input	Pydantic → 422	Input should be a valid integer
Missing resource	get_..._or_404 → 404	Skill not found
Duplicate record	check + IntegrityError handler → 409	Email is already registered
Business rule	if-check in endpoint → 400 / 409	Cannot cancel a cancelled session Mentor already has a session booked at this time
Not logged in / not allowed	dependencies → 401 / 403	Invalid or missing token

```
missing resource → 404

def get_skill_or_404(skill_id: int, db: Session) -> Skill:
    skill = db.get(Skill, skill_id)
    if skill is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Skill not found")
    return skill
```

```
app/main.py — global safety net for duplicates

@app.exception_handler(IntegrityError)
async def integrity_error_handler(request: Request, exc: IntegrityError):
    """Safety net for duplicate records that slip past the explicit checks."""
    return JSONResponse(status_code=status.HTTP_409_CONFLICT, content={"detail": "Duplicate or conflicting
    record"})
```

✔ **Proof:** steps 2, 8 and 12 show error handling live; the Postman collection has 19 negative tests (401, 403, 404, 409, 400, 422) and all pass.

REQUIREMENT 14

Environment Variables

Presented by **Member 3**

Expected: *Keep database credentials and JWT secrets in environment variables/.env.*

How we implemented it

- All secrets live in a `.env` file, never in the Python code.
- `config.py` uses **pydantic-settings** to load them; `DATABASE_URL` and `JWT_SECRET_KEY` are required, so the server will not start without them.
- `.env` is listed in `.gitignore` so it is never uploaded. We share `.env.example` as a template.

Variable	Purpose
DATABASE_URL	PostgreSQL / Supabase connection with username and password
JWT_SECRET_KEY	secret used to sign and verify tokens
JWT_ALGORITHM	signing algorithm (HS256)
ACCESS_TOKEN_EXPIRE_MINUTES	token lifetime (60)
ADMIN_EMAIL / ADMIN_PASSWORD	the admin account created on startup

```
app/config.py

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    """All secrets/config are read from environment variables or the .env file."""

    database_url: str
    jwt_secret_key: str
    jwt_algorithm: str = "HS256"
    access_token_expire_minutes: int = 60
    admin_email: str = "admin@skillswap.com"
    admin_password: str = "Admin@123"

    model_config = SettingsConfigDict(env_file=".env")


settings = Settings()
```

```
.gitignore — the real .env is never committed

.env
.venv/
__pycache__/
*.db
```

✔ **Proof:** search the code: no password or secret appears in any .py file. Changing `DATABASE_URL` in .env is all it takes to switch databases.

REQUIREMENT 15

Swagger / OpenAPI

Presented by **Member 2**

Expected: *Ensure APIs are documented and testable through FastAPI Swagger UI.*

How we implemented it

- FastAPI generates the documentation from our code: `/docs` (Swagger UI), `/redoc` and `/openapi.json`.
- We made it clear and testable: a title and description, a **tag** per router (the groups in Swagger), `response_model` for every response, `Field`/`Query` limits and descriptions that appear in the schema, and `HTTPBearer` security.
- Because of `HTTPBearer`, Swagger shows the **Authorize** button and a lock on each of the 20 protected routes, so every endpoint can be tested in the browser.

```
what feeds the documentation

app = FastAPI(
    title="SkillSwap API",
    description="Student skill sharing platform: skill profiles, mentor search, session booking and reviews.",
    version="1.0.0", ...)

router = APIRouter(prefix="/sessions", tags=["Mentoring Sessions"])        # group name in Swagger
bearer_scheme = HTTPBearer(auto_error=False)                              # Authorize button + locks
duration_minutes: int = Field(ge=30, le=180, description="30 to 180 minutes")   # shown in the schema
```

✔ **Proof:** the whole of section 2 (20 steps, 21 figures) was done in our Swagger UI.

REQUIREMENT 16

Postman Testing

Presented by **Member 5**

Expected: *Create and test the API using Postman; submit a Postman collection.*

How we implemented it

- Collection file to submit: `postman/SkillSwap.postman_collection.json`.
- **53 requests in 7 folders** that follow the real user journey, including negative tests for every error code.
- Test scripts check the status code of every response and save tokens/ids into collection variables, so the requests chain automatically.
- Emails get a timestamp and booking times are calculated, so the collection can be run again and again.

Folder	Requests	Examples
1. Auth	9	register, duplicate 409, invalid 422, login ×3, wrong password 401
2. Users & Mentors	3	my profile, no token 401, update
3. Skills	11	create, duplicate 409, student 403, invalid level 422, search, 404, update
4. Sessions	12	book, overlap 409, past 422, mentor 403, history, cancel, cancel twice 400, complete
5. Reviews	7	cancelled 400, create, duplicate 409, rating 6 422, list, update
6. Admin	8	list, student 403, stats, deactivate → login 403, reactivate, role, 404
7. Cleanup	3	delete review, skill, user (204)

```
a Postman test script

// Tests tab of "Login student"
pm.test("Status is 200", () => pm.response.to.have.status(200));
pm.collectionVariables.set("student_token", pm.response.json().access_token);
```

```
result on our PostgreSQL database

┌─────────────────────────┬──────────────────────┬──────────────────┐
│                         │             executed │           failed │
├─────────────────────────┼──────────────────────┼──────────────────┤
│                requests │                   53 │                0 │
│              assertions │                   53 │                0 │
└─────────────────────────┴──────────────────────┴──────────────────┘
```

✔ **Proof:** Postman → Import → Run collection: 53 of 53 tests pass. Command line: `npx newman run postman/SkillSwap.postman_collection.json`.

SECTION 5

Presentation Plan

Who presents what, what to say, and how to answer questions

Member	Presents	Shows on screen	Time
1	Section 1: problem statement, what an API is, the 3 roles, the journey, the 12 components. Requirements 1, 2	This PDF (sections 1.1–1.6, 4)	4 min
2	Section 2: the live workflow in Swagger, steps 1–20. Requirements 11, 15	Swagger UI in the browser	5 min
3	Sections 3.0–3.2: code layout, database, ER diagram, .env, roles. Requirements 3, 4, 14	models.py, database.py, pgAdmin	4 min
4	Sections 3.3–3.7: register/login, JWT, protection, skills, search, admin. Requirements 6, 7, 8, 9, 10	auth.py, security.py, skills.py	4 min
5	Sections 3.8–3.13: booking, cancel, history, reviews, validation, Postman. Requirements 5, 12, 13, 16	sessions.py, reviews.py, Postman runner	4 min

Member 1 — what to say

1. For requirements 1 and 2 (section 4), show the router table (29 endpoints) and the CRUD table.
2. “Our problem is SkillSwap: in every college some students can teach a skill and others want to learn it. We built the backend that connects them.”
3. Read the problem statement and explain the restaurant example for an API.
4. Introduce the 3 roles with Priya (mentor), Rahul (student) and the admin. Say why Student/Mentor became two roles (3 roles required).
5. Show the journey diagram: sign up → create skill → search → book → complete or cancel → review.
6. Show the 12-component table: “every part of the problem became an endpoint.”
7. Hand over: “Member 2 will now show this whole journey live in Swagger.”

Member 2 — what to say

1. After the demo, present requirements 11 and 15: point out the query and path parameters you used and how Swagger documents everything.
2. Open /docs and explain the colours, the lock icons and Try it out → Execute.
3. Run steps 1–5: register Priya, show 401 without a token, log in, Authorize, then 200.
4. Run steps 6–8: create a skill, then show Rahul getting 403 when he tries.
5. Run steps 9–12: search, book, and show the 409 double-booking error.
6. Run steps 13–18: complete, review, rating on the profile, cancel, history.
7. Run steps 19–20 as admin. Hand over: “Member 3 will show where all this data is stored.”

Member 3 — what to say

1. Present requirements 3, 4 and 14 (section 4): the \dt output, the ORM-to-SQL table and the .env variables.
2. Show the “office” table: each file has one job.
3. Show the ER diagram: 4 tables, one-to-many links, one review per session, cascade delete.
4. Explain the Skill model line by line and get_db (one session per request, always closed).
5. Show .env: secrets are not in the code. Open pgAdmin and show the tables and rows from the demo.
6. Show the Role enum and that the admin is created on startup. Hand over to Member 4.

Member 4 — what to say

1. Present requirements 6–10 (section 4): token example, the bcrypt rows in the database, the ownership table and the 20 protected routes.
2. Show the auth flow diagram. Explain register: 409 for duplicate email, bcrypt hash, no password in the response.
3. Explain login and the JWT: user id, role, expiry, signed with the secret.
4. Show get_current_user (401) and require_roles (403); show the permission table.
5. Show create_skill / update_skill (owner only) and search_skills (optional filters, ilike, paging).
6. Show the admin router: one line protects every admin endpoint. Hand over to Member 5.

Member 5 — what to say

1. Present requirements 5, 12, 13 and 16 (section 4): the validation table, the status-code table, the exception table and the 53/53 Postman result.
2. Show SessionCreate (future date, 30–180 minutes) and book_session with the overlap rule.
3. Show the session lifecycle and cancel_session (status change, not delete; only before start).
4. Show session_history (own vs all) and create_review (own, completed, only once).
5. Show the status-code table and the 422 example; explain HTTPException and the IntegrityError safety net.
6. Run the Postman collection: 53 of 53 tests pass. Close: “SkillSwap covers all 16 requirements. Thank you!”

Likely questions and answers

Q: Why did you choose FastAPI?
A: It validates data with Pydantic automatically, it is fast, and it creates Swagger documentation for free.

Q: What is the difference between 401 and 403?
A: 401: we do not know who you are (no token, wrong or expired token). 403: we know who you are, but your role or ownership does not allow it.

Q: What is the difference between 400 and 422?
A: 422: the data itself is wrong (type, length, range) — Pydantic finds it. 400: the data is fine but a business rule says no, like cancelling a completed session.

Q: Why do you hash passwords with bcrypt?
A: If the database is ever stolen, the real passwords are still safe. bcrypt is one-way, salted and slow on purpose.

Q: How does logout work with JWT?
A: The app simply forgets the token, and every token expires after 60 minutes. The admin can also deactivate a user, which blocks them at once because we check the database on every request.

Q: What if someone changes the role inside their token to admin?
A: The signature no longer matches, so the token is rejected with 401. Also we read the role from the database, not from the token.

Q: Why is cancel a PATCH and not a DELETE?
A: We only change the status. Keeping the row gives users a full session history and lets the admin count cancellations.

Q: How do you stop double booking?
A: Before saving we compare the new time range with every booked session of that mentor; if they overlap we return 409.

Q: Why use an ORM instead of SQL?
A: Less code, Python objects instead of rows, and it automatically protects against SQL injection.

Appendix: how to run the project

```
terminal

# 1. One time: give the postgres user a password and create the database
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';" -c "CREATE DATABASE skillswap;"

# 2. One time: create the Python environment
cd SDPP
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # edit DATABASE_URL / JWT_SECRET_KEY if needed

# 3. Every time: start the server (tables and the admin are created automatically)
source .venv/bin/activate
uvicorn app.main:app --reload
#   Swagger UI : http://127.0.0.1:8000/docs
#   Admin login: admin@skillswap.com / Admin@123

# 4. Optional: run the Postman tests from the terminal
npx newman run postman/SkillSwap.postman_collection.json
```

Common problems

Problem	Fix
password authentication failed for user “postgres”	Run step 1 again and make sure the password matches DATABASE_URL in .env
database “skillswap” does not exist	sudo -u postgres psql -c "CREATE DATABASE skillswap;"
Address already in use (port 8000)	A server is already running: pkill -f "uvicorn app.main"
401 in Swagger	Log in again and paste the new token in Authorize (tokens expire after 60 minutes)
409 when booking	That mentor already has a session at that time; choose another time
422 when booking	The time must be in the future, e.g. "2026-12-01T10:00:00Z"

Before the presentation

- PostgreSQL is running and pgAdmin shows the skillswap database.
- The server starts and /docs opens.
- Member 2 has practised steps 1–20 once (use new emails if Priya and Rahul already exist, or delete them as admin).
- Postman collection imported and run once.
- Names written on the cover page.